

WEB TECHNOLOGIES

Table of Contents

Sl. No	Topic	Pages
1	Introduction to PHP	03-19
2	Conditional Statements	20-23
3	Loops	24-25
4	String	26-28
5	Arrays	29-32
6	Functions	33-37
7	PHP Forms & Superglobals	38-42
8	PHP File Handling	43-45
9	Include & Require	46-47
10	XML	48-53
11	AJAX	54-60
12	JSON	61-65
13	Model–View–Controller (MVC)	66-69
14	PHP MySQL Database	70-81
15	Output Tracing Practice	82-86
16	Code Writing Practice	87-94
17	Analytical Questions Practice	95-96
18	MCQ practice	97-110
19	Lecture Video Link	110

INTRODUCTION TO PHP

Introduction

➤ What is PHP?

- ⇒ **PHP (Hypertext Preprocessor)** is a **server-side scripting language**.
- ⇒ Used for **web development** to create **dynamic and interactive websites**.
- ⇒ PHP code is executed on the **server**, and the result is sent to the browser as **plain HTML**.

➤ Features of PHP?

- Open-source and free.
- Easy to learn and use.
- Works with almost all servers (Apache, Nginx, IIS, etc.).
- Supports many databases (MySQL, PostgreSQL, SQLite, etc.).
- Cross-platform (runs on Windows, Linux, macOS).

PHP File Basics

- PHP files have the extension **.php**.
- A PHP script starts with **<?php** and ends with **?>**.

Example Code:

```
<?php
echo "Hello, World!";
?>
```

➤ How PHP Works:

1. Client (browser) requests a PHP page.
2. The **server executes PHP code**.
3. Server sends **HTML output** to the client.

PHP Syntax

➤ Basic Rules

1. A PHP script starts with `<?php` and ends with `?>`.
2. Each PHP statement ends with a **semicolon ;**.
3. PHP code can be written **inside HTML**.

Example Code:	Output:
<pre><?php echo "Hello, World!"; ?></pre>	Hello, World!

PHP Inside HTML

Example Code:
<pre><!DOCTYPE html> <html> <body> <h2>My First PHP Page</h2> <?php echo "This is PHP inside HTML!"; ?> </body> </html></pre>

Comments in PHP

⇒ **Comments** are notes in the code that are ignored by the browser, used to explain or temporarily disable code.

Type	Syntax	Example
Single-line	<code>// or #</code>	<pre>// This is a comment</pre>
Multi-line	<code>/* ... */</code>	<pre>/* This is a multi-line comment */</pre>

Variables

⇒ **Variable** is a name used to **store data** in memory. It acts as a container for values.

⇒ In PHP, all **variables start with \$**.

⇒ Variable names are **case-sensitive**.

➤ **How to Declare a Variable?**

Syntax:	Example Code:
<pre>\$name = "mohsin"; \$age = 20; \$salary = 30000.00;</pre>	<pre><?php \$name = "John"; // String \$age = 25; // Integer \$salary = 5000.50; // Float \$isAdmin = true; // Boolean echo "Name: \$name, Age: \$age"; ?></pre>
Rules for Naming Variables:	
• Must start with a letter or underscore <code>_</code>. • Cannot start with a digit • Can contain letters, digits, and underscores • Are case-sensitive (<code>\$name</code> $\neq$ <code>\$Name</code>).	

Variable Scope

⇒ **Local Variables** – Defined inside a function, only accessible there.

⇒ **Global Variables** – Defined outside a function, accessible globally with `global` keyword.

⇒ **Static Variables** – Retains value between function calls.

Example Code:

```
<?php  
$globalVar = "I am global";  
  
function test() {  
    global $globalVar;  
    echo $globalVar;  
}  
test(); // I am global  
?>
```

Data Types

- ⇒ A **data type** specifies what kind of **value** a variable can hold.
- ⇒ PHP automatically **detects the data type** of a variable.

Type	Example	Description
String	"Hello"	Text data
Integer	100	Whole numbers
Float	10.5	Decimal numbers
Boolean	true/false	Logical value
Array	[1,2,3]	Collection of values
Object	class Car {}	Object-oriented programming
NULL	NULL	Empty value

Type Checking Functions

Function	Description
is_string(\$var)	Checks if variable is a string
is_int(\$var)	Checks if variable is integer
is_float(\$var)	Checks if variable is float
is_bool(\$var)	Checks if variable is boolean
is_array(\$var)	Checks if variable is array
is_object(\$var)	Checks if variable is object
is_null(\$var)	Checks if variable is NULL

Output in PHP

Function	Description	Example
<code>echo</code>	Outputs text or variables	<code>echo "Hello";</code>
<code>print</code>	Similar to <code>echo</code> , returns 1	<code>print "Hello";</code>
<code>print_r()</code>	Prints arrays or objects (human-readable)	<code>print_r(\$arr);</code>
<code>var_dump()</code>	Shows type + value (debugging)	<code>var_dump(\$x);</code>

Example Code: <code>echo</code>	Output:
<pre><?php \$name = "Mohsin"; echo "Hello, " . \$name . "!"; // Concatenation ?></pre>	Hello, Mohsin!
Example Code: <code>print</code>	Output:
<pre><?php \$age = 22; if (print "Age: \$age") { echo " (Printed successfully)"; } ?></pre>	Age: 22 (Printed successfully)
Example Code: <code>print_r()</code>	Output:
<pre><?php \$fruits = array("Apple", "Banana", "Mango"); print_r(\$fruits); ?></pre>	Array ([0] => Apple [1] => Banana [2] => Mango)
Example Code: <code>var_dump()</code>	Output:
<pre><?php \$price = 99.50; \$active = true; var_dump(\$price); var_dump(\$active); ?></pre>	float(99.5) bool(true)

Type Conversion

➔ Type Juggling (Automatic Conversion):

⇒ PHP automatically converts data types when needed.

Example Code:

```
<?php
$x = "10";    // string
$y = 5;       // integer
$z = $x + $y; // string "10" is converted to integer 10

echo $z; // Output: 15
echo gettype($z); // Output: integer
?>
```

➔ Type Casting (Manual Conversion):

⇒ We can explicitly convert one type to another using **casting**.

Example Code:

```
<?php
$a = "100";
$b = (int)$a; // Convert string to integer
$c = 12.34;
$d = (string)$c; // Convert float to string

echo $b; echo " (" . gettype($b) . ")<br>";
echo $d; echo " (" . gettype($d) . ")";
?>
```

➔ Functions for Type Conversion:

Function	Description	Example	Result
<code>strval()</code>	Convert to string	<code>strval(123)</code>	"123"
<code>intval()</code>	Convert to integer	<code>intval("25.6")</code>	25
<code>floatval()</code>	Convert to float	<code>floatval("10.5")</code>	10.5
<code>boolval()</code>	Convert to boolean	<code>boolval(0)</code>	false
<code>settype()</code>	Force a variable into another type	<code>settype(\$x, "string")</code>	changes variable type

Operators

⇒ Operators are **special symbols** used to perform operations on **values or variables**.

Example Code:	Output:
<pre><?php \$a = 5; \$b = 2; \$sum = \$a + \$b; // + is the operator echo "sum:" . \$sum; ?></pre>	Sum: 7

Types of Operators

a) Arithmetic Operators:

⇒ Used for **mathematical calculations**.

Operator	Description	Example	Result
+	Addition	5 + 2	7
-	Subtraction	5 - 2	3
*	Multiplication	5 * 2	10
/	Division	5 / 2	2.5
%	Modulus (remainder)	5 % 2	1
**	Exponentiation	5 ** 2	25
++	Increment (+1)	\$a++	6
--	Decrement (-1)	\$b--	1

b) Assignment Operators:

⇒ Used to **assign values** to variables.

Operator	Description	Example	Same As
=	Assign	\$x = 10	\$x = 10
+=	Add and assign	\$x += 5	\$x = \$x+5
-=	Subtract and assign	\$x -= 5	\$x = \$x-5
*=	Multiply and assign	\$x *= 5	\$x = \$x*5
/=	Divide and assign	\$x /= 5	\$x = \$x/5
%=	Modulus and assign	\$x %= 3	\$x = \$x%3

c) Comparison Operators:

⇒ Used to **compare two values**. Result is **true** or **false**.

Operator	Description	Example	Result
<code>==</code>	Equal	<code>5 == "5"</code>	true
<code>===</code>	Identical (equal + same type)	<code>5 === "5"</code>	false
<code>!=</code>	Not equal	<code>5 != 3</code>	true
<code><></code>	Not equal	<code>5 <> 3</code>	true
<code>!==</code>	Not identical	<code>5 !== "5"</code>	true
<code>></code>	Greater than	<code>5 > 3</code>	true
<code><</code>	Less than	<code>5 < 3</code>	false
<code>>=</code>	Greater than or equal to	<code>5 >= 5</code>	true
<code><=</code>	Less than or equal to	<code>3 <= 5</code>	true
<code><=></code>	Spaceship (returns -1, 0, or 1)	<code>10 <=> 5</code>	1

d) Logical Operators:

⇒ Used for **Boolean logic**.

Operator	Example Code	Result (if <code>\$x = 7</code>)
<code>&&</code>	<code>(\$x > 5 && \$x < 10)</code>	true (7 is between 5 and 10)
<code> </code>	<code>(\$x > 5 \$x < 10)</code>	true (7 is between 5 and 10)
<code>!</code>	<code>!(\$x > 5)</code>	false (because <code>\$x > 5</code> is true, negation = false)
<code>and</code>	<code>(\$x > 5 and \$x < 10)</code>	true (same as <code>&&</code>)
<code>or</code>	<code>(\$x > 5 or \$x < 10)</code>	true (at least one condition is true)
<code>xor</code>	<code>(\$x > 5 xor \$x < 10)</code>	false (both are true → xor needs only one true)

e) String Operators:

⇒ Mainly **concatenation**.

Operator	Description	Example	Result
<code>.</code>	Concatenation operator (joins strings)	<code>"Hello" . " World"</code>	Hello World
<code>.=</code>	Concatenation assignment (append string)	<code>\$txt = "Hello"; \$txt .= " PHP";</code>	Hello PHP

Example Code:	Output:
<pre> <?php \$first = "Hello"; \$second = "World"; echo \$first . " " . \$second; echo "
"; \$text = "PHP"; \$text .= " is awesome!"; echo \$text; ?> </pre>	Hello World PHP is awesome!

f) Increment / Decrement Operators:

Operator	Description	Example	Result
++\$x	Pre-increment (increase then use)	<code>\$x=5; ++\$x;</code>	6
\$x++	Post-increment (use then increase)	<code>\$x=5; \$x++;</code>	5 (then \$x=6)
--\$x	Pre-decrement (decrease then use)	<code>\$x=5; --\$x;</code>	4
\$x--	Post-decrement (use then decrease)	<code>\$x=5; \$x--;</code>	5 (then \$x=4)

f) Array Operators:

Operator	Description	Example	Result
+	Union (combine arrays)	<code>\$a + \$b</code>	Array union
==	Equality (same key/value pairs)	<code>\$a == \$b</code>	true/false
===	Identity (same key/value + order + type)	<code>\$a === \$b</code>	true/false
!=	Inequality	<code>\$a != \$b</code>	true/false
<>	Inequality	<code>\$a <> \$b</code>	true/false
!==	Non-identity	<code>\$a !== \$b</code>	true/false

How to See the Data Type

⇒ **gettype()** → Returns the type of a variable as a string.

Example Code:	Output:
<pre><?php \$x = 100; echo gettype(\$x); // integer ?></pre>	integer

⇒ **var_dump()** → Shows the **type** and **value** (best for debugging).

Example Code:	Output:
<pre><?php \$y = 12.5; var_dump(\$y); ?></pre>	float(12.5)

Keywords

➔ **Keywords** are reserved words in PHP. We cannot use them as variable names or identifiers.

List of Common PHP Keywords:

Keywords	Keywords	Keywords	Keywords	Keywords	Keywords
break	case	catch	class	const	continue
die	default	do	else	elseif	exit
extends	finally	for	function	if	include
include_once	require	require_once	new	return	static
switch	throw	try	var	while	unset
isset	empty	global	abstract	final	public
private	protected	clone	trait	interface	namespace
use	echo	print	yield		

- ✂ **Problem1:** Write a PHP program that assign values to variables num1, num2 and perform addition operations. Also print the results.

Code:	Output:
<pre><?php \$num1 = 10; \$num2 = 5; \$sum = \$num1 + \$num2; echo "Sum: ". \$sum; ?></pre>	Sum: 15

- ✂ **Problem2:** Write a PHP program that assign values to variables num1, num2 and perform subtraction operations. Also print the results.

Code:	Output:
<pre><?php \$num1 = 10; \$num2 = 5; \$sub = \$num1 - \$num2; echo "Sub: ". \$sub; ?></pre>	Sub: 5

- ✂ **Problem3:** Write a PHP program that assign values to variables num1, num2 and perform multiplication & division operations. Also print the results.

Code:	Output:
<pre><?php \$num1 = 10; \$num2 = 5; \$multi = \$num1 * \$num2; \$div = \$num1 / \$num2; echo "Multi: ". \$multi; echo "
" echo "Div: ". \$div; ?></pre>	Mult: 50 Div: 2.0

Input in PHP

➔ In PHP, input usually comes from **users via HTML forms**. PHP can access this input using **superglobal arrays**:

Method	Superglobal Array	Description
GET	<code>\$_GET</code>	Retrieves data from URL query string (<code>?name=value</code>)
POST	<code>\$_POST</code>	Retrieves data sent via HTML form using <code>method="post"</code>
REQUEST	<code>\$_REQUEST</code>	Retrieves data from both GET and POST
COOKIE	<code>\$_COOKIE</code>	Retrieves data stored in cookies
SERVER	<code>\$_SERVER</code>	Retrieves server info (like headers, IP, URL, etc.)

Example Code: HTML Form + PHP Input

HTML Form: (input.html)

```
<form action="process.php" method="post">
  Name: <input type="text" name="username"><br>
  Age: <input type="number" name="age"><br>
  <input type="submit" value="Submit">
</form>
```

PHP Script: (process.php)

```
<?php
// Using $_POST to get input from form
$name = $_POST['username'];
$age = $_POST['age'];

echo "Hello, " . $name . "! You are " . $age . " years old.";
?>
```

Output:

Hello, Mohsin! You are 23 years old.

Example Code: HTML Form + with method="get"

HTML Form: (input.html)	
<pre><form method="get" action="process.php"> Name: <input type="text" name="name">
 Age: <input type="number" name="age">
 <input type="submit" value="Submit"> </form></pre>	
PHP Script: (process.php)	
<pre><?php \$name = \$_GET['name']; \$age = \$_GET['age']; echo "Hello, \$name! You are \$age years old."; ?></pre>	
Output:	After submitting, the browser URL becomes:
Hello, Mohsin! You are 23 years old.	process.php?name=mohsin&age=23

Notes:

- ⇒ Use POST for sensitive data (passwords, etc.) because GET shows data in URL.
- ⇒ Always sanitize user input to prevent security risks:

Without sanitize:	Output:
<pre><?php \$name = \$_POST['name']; \$age = \$_POST['age']; echo "Hello, \$name! You are \$age years old."; ?></pre>	Name: <h2>Mohsin</h2> Age: 20 <input type="submit" value="Submit"/>
	Hello, Mohsin ! You are 20 years old.
With sanitize:	Output:
<pre><?php \$name = htmlspecialchars(\$_POST['name']); \$age = intval(\$_POST['age']); echo "Hello, \$name! You are \$age years old."; ?></pre>	Name: <h2>Mohsin</h2> Age: 20 <input type="submit" value="Submit"/>
	Hello, <h2>Mohsin</h2>! You are 20 years old.

PHP Input Methods Table

Method	Superglobal	Example Usage	Example Code	Notes
GET	<code>\$_GET</code>	Get values from URL query string	<code>\$_GET['name']</code>	Data visible in URL, less secure
POST	<code>\$_POST</code>	Get values from form submission	<code>\$_POST['name']</code>	Data hidden in URL, more secure
REQUEST	<code>\$_REQUEST</code>	Get values from both GET & POST	<code>\$_REQUEST['name']</code>	Not recommended for sensitive input (mixes both)
COOKIE	<code>\$_COOKIE</code>	Get values from cookies	<code>\$_COOKIE['user']</code>	Stored on client's browser
SESSION	<code>\$_SESSION</code>	Get values from session variables	<code>\$_SESSION['user']</code>	Stored on server, more secure
SERVER	<code>\$_SERVER</code>	Get server/environment info	<code>\$_SERVER['REMOTE_ADDR']</code>	Useful for IP, headers, etc.
FILES	<code>\$_FILES</code>	Handle file uploads	<code>\$_FILES['file']['name']</code>	Used when <code>enctype="multipart/form-data"</code>

Example Code: HTML Form + PHP Input

HTML Form: (input.html)

```
<!-- GET Form -->
<h3>GET Method</h3>
<form action="process.php" method="get">
  Name: <input type="text" name="user">
  <input type="submit" value="Submit">
</form>

<!-- POST Form -->
<h3>POST Method</h3>
<form action="process.php" method="post">
  Name: <input type="text" name="user">
  <input type="submit" value="Submit">
</form>

<!-- File Upload Form -->
<h3>File Upload</h3>
<form action="process.php" method="post" enctype="multipart/form-data">
  Select file: <input type="file" name="myfile">
  <input type="submit" value="Upload File">
</form>
```

PHP Script: (process.php)

```
<?php
// GET
if (isset($_GET['user'])) {
    echo "GET: " . htmlspecialchars($_GET['user']) . "<br>";
}

// POST
if (isset($_POST['user'])) {
    echo "POST: " . htmlspecialchars($_POST['user']) . "<br>";
}

// REQUEST
if (isset($_REQUEST['user'])) {
    echo "REQUEST: " . htmlspecialchars($_REQUEST['user']) . "<br>";
}

// COOKIE
if (isset($_COOKIE['user'])) {
    echo "COOKIE: " . $_COOKIE['user'] . "<br>";
}

// SESSION (must call session_start())
session_start();
$_SESSION['user'] = "Mohsin";
echo "SESSION: " . $_SESSION['user'] . "<br>";

// SERVER
echo "SERVER (Your IP): " . $_SERVER['REMOTE_ADDR'] . "<br>";
?>
```

Note: When you upload a file in PHP, the structure of `$_FILES` looks like this:

```
$_FILES['myfile'] = array(
    'name'      => 'photo.jpg',          // Original filename (from user's computer)
    'type'      => 'image/jpeg',         // File type (MIME type)
    'tmp_name'  => '/tmp/phpYzdqkD',     // Temporary location on the server
    'error'     => 0,                    // Upload status (0 = no error)
    'size'      => 15234                  // File size in bytes
);
```

That's the reasons We use `$_FILES['myfile']['name']` because the file input returns multiple pieces of info (not just one value like `$_POST`),

Why Use `isset()` in PHP

- ⇒ The `isset()` function is used to **check if a variable is set and is not NULL**.
- Returns `true` → if the variable exists **and** is not `NULL`.
 - Returns `false` → if the variable **does not exist** or is `NULL`.

Syntax:

```
<?php
isset($variable);
isset($var1, $var2, $var3); // can check multiple variables at once
?>
```

➤ Why it's important in forms

- ⇒ When you use `$_POST`, `$_GET`, or `$_FILES`, the user may not have submitted the field, so the variable might not exist.
- ⇒ If you try to access it **without checking**, PHP will give a “undefined index” warning.

Example without <code>isset()</code>	Output (if form not submitted):
<pre><?php echo \$_POST['username']; ?></pre>	Notice: Undefined index: username
Example with <code>isset()</code>	Output (if form not submitted):
<pre><?php if (isset(\$_POST['username'])) { echo "Hello, " . \$_POST['username']; } else { echo "No username provided."; } ?></pre>	No username provided.

CONDITIONAL STATEMENTS

Conditional Statements

- Conditional statements are used to **make decisions** in a program.
- They check a **condition** (true/false) and execute code accordingly.

Common String Operations:

if Statement:

```
$age = 20;

if ($age >= 18) {
    echo "You are an adult.";
}
```

if...else Statement:

```
<?php
$age = 16;

if ($age >= 18) {
    echo "You are an adult.";
} else {
    echo "You are a minor.";
}
?>
```

if...else if...else Statement:

```
<?php
$marks = 75;

if ($marks >= 80) {
    echo "Grade: A";
} elseif ($marks >= 60) {
    echo "Grade: B";
} else {
    echo "Fail";
}
?>
```

Notes:

- Conditions use **comparison operators** (==, !=, >, <, >=, <=)

Nested Conditional Statements

➔ A **nested conditional** means using one if or else block **inside another**.

It allows for **more complex decision making**.

Nested if:

```
$age = 20;
$citizen = true;
if ($age >= 18) {
    if ($citizen) {
        echo "You are eligible to vote.";
    } else {
        echo "You must be a citizen to vote.";
    }
} else {
    echo "You are not old enough to vote.";
}
```

Output (if \$age=20 and \$citizen=true):

You are eligible to vote.

switch Statement: Used when comparing a variable with **multiple values**.

```
<?php
$day = "Monday";

switch ($day) {
    case "Monday":
        echo "Start of the week!";
        break;
    case "Friday":
        echo "Weekend is near!";
        break;
    case "Sunday":
        echo "Holiday!";
        break;
    default:
        echo "Regular day.";
}
?>
```

Practice Problem

✂ **Problem1:** Write a PHP Script to check whether a number entered by the user is **odd** or **even**, and print the result.

Code:	Output:
<pre><?php if (isset(\$_POST['num'])) { \$num = (int)\$_POST['num']; // Convert input to integer if (\$num % 2 === 0) { echo "Even"; } else { echo "Odd"; } } else { echo "Please enter a number."; } ?></pre>	Enter a number: 10 Even
	Enter a number: 5 Odd

✂ **Problem2:** Write a PHP Script to **grade students based on their marks** using the following criteria:

- If marks are **90 or above**, grade = "A"
 - If marks are **80 to 89**, grade = "B"
 - If marks are **70 to 79**, grade = "C"
 - If marks are **below 70**, grade = "D"
- Then print the grade.

Code:	Output:
<pre><?php if (isset(\$_POST['marks'])) { \$marks = (int)\$_POST['marks']; \$grade = ""; if (\$marks >= 90) { \$grade = "A"; } elseif (\$marks >= 80) { \$grade = "B"; } elseif (\$marks >= 70) { \$grade = "C"; } else { \$grade = "D"; } echo "Grade: " . \$grade; } else { echo "Please enter the marks."; } ?></pre>	Enter student's marks: 73 Grade: C

LOOPS

Loops

➔ **Loops** are used to execute a block of code repeatedly.

Types of Loops:

for Loop: iterate over a sequence	while Loop: runs while a condition is True
<pre><?php for (\$i = 1; \$i <= 5; \$i++) { echo "Count: \$i
"; } ?></pre>	<pre><?php \$i = 1; while (\$i <= 5) { echo "Number: \$i
"; \$i++; } ?></pre>
do...while Loop: Runs the code once before checking the condition.	foreach Loop: Special loop for arrays.
<pre><?php \$i = 1; do { echo "Value: \$i
"; \$i++; } while (\$i <= 3); ?></pre>	<pre><?php \$fruits = ["Apple", "Banana", "Mango"]; foreach (\$fruits as \$fruit) { echo "\$fruit
"; } ?></pre>
break Statement: Stops the loop immediately.	continue Statement: Skips the current iteration and moves to the next.
<pre><?php for (\$i = 1; \$i <= 5; \$i++) { if (\$i == 3) { break; // exit the loop when \$i is 3 } echo "Number: \$i
"; } ?></pre>	<pre><?php for (\$i = 1; \$i <= 5; \$i++) { if (\$i == 3) { continue; // skip the iteration when \$i is 3 } echo "Number: \$i
"; } ?></pre>

STRING

Strings

➔ A **string** is a sequence of characters inside **single quotes (')**, **double quotes (")**.

Example Code:

```
$str1 = "Hello";  
$str2 = 'World';
```

Creating Strings:

```
$str1 = "Hello World"; // Double quotes  
$str2 = 'Hello PHP';   // Single quotes  
  
echo $str1;
```

String Concatenation: Use the **.** (dot) operator to join strings.

```
$first = "Hello";  
$last = "World";  
  
$full = $first . " " . $last;  
echo $full; // Hello World
```

Concatenation assignment: .=

```
$txt = "Hello";  
$txt .= " PHP";  
echo $txt; // Hello PHP
```

Common String Methods:

Function	Description	Example
<code>strlen(\$str)</code>	Returns string length	<code>strlen("Hello")</code> → 5
<code>str_word_count(\$str)</code>	Counts words in a string	<code>str_word_count("Hello World")</code> → 2
<code>strrev(\$str)</code>	Reverses a string	<code>strrev("Hello")</code> → "olleH"
<code>strpos(\$str, \$find)</code>	Finds position of substring	<code>strpos("Hello", "e")</code> → 1
<code>str_replace(\$search, \$replace, \$str)</code>	Replaces text	<code>str_replace("World", "PHP", "Hello World")</code> → "Hello PHP"
<code>strtoupper(\$str)</code>	Converts to uppercase	<code>strtoupper("hello")</code> → "HELLO"
<code>strtolower(\$str)</code>	Converts to lowercase	<code>strtolower("HELLO")</code> → "hello"
<code>trim(\$str)</code>	Removes spaces from start/end	<code>trim(" Hello ")</code> → "Hello"
<code>substr(\$str, \$start, \$length)</code>	Extracts part of string	<code>substr("Hello", 1, 3)</code> → "ell"

Practice Problem

🔗 **Problem1:** Write a PHP script to input user's first name & print its length.

Code:	Output:
<pre>if (isset(\$_POST['first_name'])) { \$first_name = \$_POST['first_name']; \$length = strlen(\$first_name); // Get length of string echo "Length of first name is: " . \$length; } else { echo "Please enter your first name."; }</pre>	Enter first name: Mohsin Length of first name is: 6

🔗 **Problem2:** Write a JavaScript program to initialize a string and find how many times the character '\$' appears in it. text = "He bought a pen for \$5 and a book for \$10, total \$15."

Code:	Output:
<pre>\$text = "He bought a pen for \$5 and a book for \$10, total \$15."; \$count = substr_count(\$text, '\$'); // Count occurrences of '\$' echo "The character '\$' appears " . \$count . " time(s).";</pre>	The character '\$' appears 3 time(s).

🔗 **Problem3:** Prompt the user to enter their full name. Generate a username for them based on the input. Start username with @, followed by their full name and ending with the fullname length. eg: user name = "mohsineli", username should be "@mohsineli9"

Code:	Output:
<pre>if (isset(\$_POST['full_name'])) { \$fullName = \$_POST['full_name']; // Get user input \$username = "@" . \$fullName . strlen(\$fullName); echo "Your username: " . \$username; } else { echo "Please enter your full name."; }</pre>	Your username: @mohsineli9

ARRAYS

Arrays

⇒ An **array** is a special variable that can hold **multiple values** in a single name.

- Array elements can be of **different data types**.
- Indexing starts at **0**.

Syntax:	Example:
<pre>\$VariableName = ["item1", "item2"]</pre>	<pre>\$fruits = ["Apple", "Banana", "Mango"];</pre>
Creating Arrays: Indexed Arrays	Associative Arrays: Arrays with custom keys instead of numeric indexes.
<pre>\$fruits = ["Apple", "Banana", "Mango"]; \$fruits = array("Apple", "Banana", "Mango"); \$fruits = []; // empty array</pre>	<pre>\$ages = ["Nihan" => 25, "Alif" => 30, "Jam" => 20];</pre>
Accessing Array Elements:	
<pre>\$fruits = ["Apple", "Banana", "Mango"]; echo \$fruits[0]; // Apple echo \$fruits[2]; // Mango</pre>	<pre>\$ages = ["Nihan" => 25, "Alif" => 30, "Jam" => 20]; echo \$ages["alif"]; // 30</pre>
Changing Array Elements:	
<pre>\$fruits = ["Apple", "Banana", "Mango"]; \$fruits[1] = "Orange"; // Change "Banana" to "Orange"</pre>	<pre>\$ages = array("Nihan" => 25, "Alif" => 30, "Jam" => 20); \$ages["Alif"] = 25; // Change alif's age</pre>

Multidimensional Arrays

Example code:	Output:
<pre>\$matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]; \$matrix[1][2] = 10; // Change element 6 to 10 echo \$matrix[1][2]; // 6</pre>	6

Common Array Functions:

Function	Description	Example
<code>count(\$arr)</code>	Returns number of elements	<code>count(\$fruits) → 3</code>
<code>array_push(\$arr, \$value)</code>	Adds element at end	<code>array_push(\$fruits, "Orange")</code>
<code>array_pop(\$arr)</code>	Removes last element	<code>array_pop(\$fruits)</code>
<code>array_shift(\$arr)</code>	Removes first element	<code>array_shift(\$fruits)</code>
<code>array_unshift(\$arr, \$value)</code>	Adds element at start	<code>array_unshift(\$fruits, "Lemon")</code>
<code>array_merge(\$arr1, \$arr2)</code>	Merges arrays	<code>array_merge(\$arr1, \$arr2)</code>
<code>in_array(\$value, \$arr)</code>	Checks if value exists	<code>in_array("Apple", \$fruits) → true</code>
<code>array_key_exists(\$key, \$arr)</code>	Checks if key exists	<code>array_key_exists("John", \$ages) → true</code>
<code>array_slice(\$arr, \$start, \$length)</code>	Returns part of array	<code>array_slice(\$fruits, 1, 2)</code>
<code>array_search(\$value, \$arr)</code>	Returns key of value	<code>array_search("Mango", \$fruits) → 2</code>
<code>sort(\$arr)</code>	Sorts array ascending	<code>sort(\$fruits)</code>
<code>rsort(\$arr)</code>	Sorts array descending	<code>rsort(\$fruits)</code>
<code>asort(\$arr)</code>	Sorts associative array by value	<code>asort(\$ages)</code>
<code>ksort(\$arr)</code>	Sorts associative array by key	<code>ksort(\$ages)</code>

Practice Problem

✂ **Problem1:** For a given array with marks of students → [85, 97, 44, 37, 76, 60] Find the average marks of the entire class.

Code:	Output:
<pre>\$marks = [85, 97, 44, 37, 76, 60]; \$sum = 0; for (\$i = 0; \$i < count(\$marks); \$i++) { \$sum += \$marks[\$i]; } \$average = \$sum / count(\$marks); echo "Average Marks: " . \$average;</pre>	Average Marks: 66.5

- ✂ **Problem2:** For a given array with prices of 5 items → [250, 645, 300, 900, 50] All items have an offer of 10% OFF on them. Change the array to store final price after applying offer.

Code:	Output:
<pre>\$prices = [250, 645, 300, 900, 50]; for (\$i = 0; \$i < count(\$prices); \$i++) { \$discount = \$prices[\$i] * 0.10; \$prices[\$i] = \$prices[\$i] - \$discount; } echo "Final Prices after 10% OFF: "; print_r(\$prices);</pre>	<pre>Final Prices after 10% OFF: Array ([0] => 225 [1] => 580.5 [2] => 270 [3] => 810 [4] => 45)</pre>

- ✂ **Problem3:** Create an array to store companies → “Bloomberg”, “Microsoft”, “Uber”, “Google”, “IBM”, “Netflix”

- Remove the first company from the array
- Remove Uber & Add Ola in its place
- Add Amazon at the end

Code:	Output:
<pre>\$companies = ["Bloomberg", "Microsoft", "Uber", "Google", "IBM", "Netflix"]; // a. Remove the first company array_shift(\$companies); // b. Remove "Uber" & Add "Ola" in its place \$uberIndex = array_search("Uber", \$companies); // find index if (\$uberIndex !== false) { array_splice(\$companies, \$uberIndex, 1, "Ola"); // replace } // c. Add "Amazon" at the end array_push(\$companies, "Amazon"); print_r(\$companies);</pre>	<pre>Array ([0] => Microsoft [1] => Ola [2] => Google [3] => IBM [4] => Netflix [5] => Amazon)</pre>

FUNCTIONS

Functions

➔ A **function** is a block of reusable code that performs a specific task.

Define a Function:	Call a Function:
<pre>function function_name(){ echo "Hello World!"; }</pre>	<pre>function_name();</pre>
Example code:	Output:
<pre>function greet() { echo "Hello, World!"; } // Call the function greet();</pre>	Hello, World!
Notes: Use function to define a function.	

Function with Parameters

⇒ A Parameters allow passing **values to a function**.

Example code:	Output:
<pre>function greet(\$name) { echo "Hello, \$name!" . "
"; } greet("Alif"); greet("Jam");</pre>	Hello, Alif! Hello, Jam!
Notes: Functions help make code modular and reusable .	

Function with Return Value

⇒ **return** sends a **value back** to the caller.

Example code:	Output:
<pre>function add(\$a, \$b) { return \$a + \$b; } \$result = add(5, 10); echo \$result; // 15</pre>	15

Default Parameters

Example code:	Output:
<pre>function greet(\$name = "Guest") { echo "Hello, \$name!" . "
"; } greet(); greet("Nihan");</pre>	Hello, Guest! Hello, Nihan!

Variable Scope

⇒ **Local Scope:** Variables inside a function are **not accessible outside**.

⇒ **Global Scope:** Use global keyword to access global variables.

Example code:	Output:
<pre>\$globalVar = 10; function showVar() { global \$globalVar; echo \$globalVar; } showVar(); // 10</pre>	10

Static Variables

⇒ Variables that **retain their value** between function calls.

Example code:	Output:
<pre>function counter() { static \$count = 0; \$count++; echo \$count . "
"; } counter(); // 1 counter(); // 2 counter(); // 3</pre>	<pre>1 2 3</pre>

Built-in PHP Functions

Function	Description	Example
<code>strlen(\$str)</code>	Returns string length	<code>strlen("Hello")</code> → 5
<code>array_push(\$arr, \$val)</code>	Adds element to array	<code>array_push(\$fruits, "Apple")</code>
<code>sort(\$arr)</code>	Sorts array in ascending order	<code>sort(\$numbers)</code>
<code>date(\$format)</code>	Returns current date/time	<code>date("Y-m-d")</code>
<code>str_replace(\$search,\$replace,\$str)</code>	Replaces string text	<code>str_replace("World", "PHP", "Hello World")</code>

Practice Problem

🔗 **Problem1:** Create a function using the “function” keyword that takes a String as an argument & returns the number of vowels in the string.

Code:	Output:
<pre>function countVowels(\$str) { \$count = 0; \$vowels = "aeiouAEIOU"; for (\$i = 0; \$i < strlen(\$str); \$i++) { if (strpos(\$vowels, \$str[\$i]) !== false) { \$count++; } } return \$count; } echo countVowels("Hello World"); // Output: 3</pre>	3

🔗 **Problem2:** Perform the same task using for each loops

Code:	Output:
<pre>function countVowels(\$str) { \$count = 0; \$vowels = "aeiouAEIOU"; \$chars = str_split(\$str); foreach (\$chars as \$char) { if (strpos(\$vowels, \$char) !== false) { \$count++; } } return \$count; } echo countVowels("Hello World"); // Output: 3</pre>	3

PHP FORMS & SUPERGLOBALS

PHP Forms

- **Forms** collect user input and send it to the server.
- Commonly used with **<form> HTML tag**.
- Data is sent to PHP using **GET** or **POST** methods.

Example Code: HTML Form + PHP Input

HTML Form: (index.html)

```
<form method="post" action="welcome.php">
  Name: <input type="text" name="username">
  <input type="submit" value="Submit">
</form>
```

PHP Superglobals

- Superglobals are **built-in variables** that are **always available** in PHP.

a) \$_GET:

- Used to collect form data sent with **method="get"**.
- Data is visible in the **URL**.

PHP Script: (welcome.php)	URL:
<pre>// welcome.php echo "Welcome " . \$_GET["username"];</pre>	welcome.php?username=Mohsin

b) \$_POST:

- Used to collect form data sent with **method="post"**.
- Data is **not visible in URL** (more secure).

PHP Script: (welcome.php)
<pre>// welcome.php echo "Welcome " . \$_POST["username"];</pre>

C) \$_REQUEST:

- Can collect data from both \$_GET and \$_POST

PHP Script: (welcome.php)

```
// welcome.php
echo "Hello " . $_REQUEST["username"];
```

D) \$_SERVER:

- Contains **information about the server and execution environment.**

PHP Script: (welcome.php)

```
echo $_SERVER['PHP_SELF']; // Current script name
echo $_SERVER['SERVER_NAME']; // Server name
echo $_SERVER['REQUEST_METHOD']; // GET or POST
```

D) \$_FILES:

- Used for **file uploads.**

HTML Form: (index.html)

```
<form action="upload.php" method="post" enctype="multipart/form-data">
  Select file: <input type="file" name="myfile">
  <input type="submit" value="Upload">
</form>
```

PHP Script: (upload.php)

```
// upload.php
echo "File Name: " . $_FILES["myfile"]["name"];
```


E) \$_SESSION:

- A **session** stores information (variables) that can be used across **multiple pages**.
- Unlike cookies, session data is stored on the **server**.
- Each visitor gets a **unique session ID**.

Starting a Session: To use sessions, call `session_start()` at the **beginning of the script**.

```
<?php
session_start();
?>
```

Creating Session Variables:

```
session_start();
$_SESSION["username"] = "Mohsin";
$_SESSION["email"] = "mohsin@example.com";

echo "Session variables are set.";
```

Accessing Session Variables:

```
session_start();
echo "Username: " . $_SESSION["username"];
echo "Email: " . $_SESSION["email"];
```

Destroying Session Variables:

Unset a single variable:

```
unset($_SESSION["username"]);
```

Destroy the whole session:

```
session_destroy();
```

Example: Login with Sessions

login.php

```
<?php
session_start();
$_SESSION["user"] = "Mohsin";
?>
```

profile.php

```
<?php
session_start();
echo "Welcome, " . $_SESSION["user"];
?>
```

E) \$_COOKIE:

- A **cookie** is a small piece of data stored on the **client's browser**.
- Used to remember information between page reloads or visits (e.g., username, theme).
- Unlike sessions, cookies are stored on the **user's computer**.

Setting a Cookie: Use the `setcookie()` function:

```
<?php
setcookie(name, value, expire, path, domain, secure, httponly);
?>
```

Accessing a Cookie:

```
if(isset($_COOKIE["username"])) {
    echo "Welcome back, " . $_COOKIE["username"];
} else {
    echo "No cookie found!";
}
```

Modifying a Cookie: Just call `setcookie()` again with the same name and a new value.

```
setcookie("username", "NIAHN", time() + (86400 * 30), "/");
echo "Before Cookie updated!";

setcookie("username", "JAM", time() + (86400 * 30), "/");
echo "After Cookie updated!";
```

Deleting a Cookie: To delete a cookie, set its expiration date to the **past**:

```
setcookie("username", "", time() - 3600, "/");
echo "Cookie deleted!";
```

Example: Remember Me Login

login.php	welcome.php
<pre>if(isset(\$_POST['username'])) { \$user = \$_POST['username']; setcookie("username", \$user, time() + (86400 * 30), "/"); echo "Hello, " . \$user . "! Cookie saved."; }</pre>	<pre>if(isset(\$_COOKIE["username"])) { echo "Welcome back, " . \$_COOKIE["username"]; } else { echo "Please log in first."; }</pre>

PHP FILE HANDLING

File Handling

- PHP allows you to **create, open, read, write, and delete files** on the server.
- Functions are mainly based on **fopen(), fread(), fwrite(), fclose()**.

➔ Opening a File: Use **fopen(filename, mode)**

- **Modes:**

- **"r"** → Read only
- **"w"** → Write (erases file)
- **"a"** → Append (writes at end)
- **"x"** → Create new file (error if exists)
- **"r+", "w+", "a+", "x+" → Read & Write**

Example Code:

Opening a File:

```
<?php
$file = fopen("test.txt", "r") or die("Unable to open file!");
?>
```

Reading a File:

```
$file = fopen("test.txt", "r") or die("Unable to open file!");
echo fread($file, filesize("test.txt")); // Reads whole file
fclose($file);
```

Note:

- ⇒ **fgets(\$file)** → reads one line
- ⇒ **fgetc(\$file)** → reads one character

Writing to a File:

```
<?php
$file = fopen("newfile.txt", "w") or die("Unable to open file!");
$txt = "Hello World\n";
fwrite($file, $txt);
$txt = "Welcome to PHP File Handling\n";
fwrite($file, $txt);
fclose($file);
?>
```

Appending to a File:

```
<?php
$file = fopen("newfile.txt", "a") or die("Unable to open file!");
fwrite($file, "This line is appended.\n");
fclose($file);
?>
```

Deleting a File:

```
<?php
if (file_exists("oldfile.txt")) {
    unlink("oldfile.txt");
    echo "File deleted successfully!";
} else {
    echo "File not found.";
}
?>
```

Checking File Existence:

```
<?php
if (file_exists("test.txt")) {
    echo "File exists!";
} else {
    echo "File does not exist.";
}
?>
```

Example Code:

Read File Line by Line:

```
<?php
$file = fopen("data.txt", "r") or die("Unable to open file!");
while(!feof($file)) {
    echo fgets($file) . "<br>";
}
fclose($file);
?>
```

**INCLUDE &
REQUIRE**

Include & Require

- ⇒ Both are used to **insert the content of one PHP file into another** before execution.
- ⇒ Helps in **code reusability** (e.g., headers, footers, database connections).

include Statement: If the file is **not found**, it **throws a warning** but the script **continues execution**.

```
<?php
include "header.php"; // Adds header file
echo "Welcome to my homepage!";
?>
```

require Statement: If the file is **not found**, it **throws a fatal error** and the script **stops execution**.

```
<?php
require "config.php"; // Adds config file
echo "Database connected successfully!";
?>
```

Difference Between include and require

Feature	include	require
Missing file effect	Warning	Fatal Error
Script continues?	Yes	No
Usage case	Optional files	Mandatory files

➤ include_once and require_once

They prevent the same file from being included multiple times.

```
<?php
include_once "functions.php"; // File included only once
require_once "db.php";       // File required only once
?>
```

XML

Introduction to XML

- XML = eXtensible Markup Language
- Used to **store and transport data**.
- Looks similar to **HTML**, but it is **for data, not for display**.
- Self-descriptive and can represent **structured information**.

Basic Structure of XML

Example:

```
<?xml version="1.0" encoding="UTF-8"?>
<note>
  <to>Tony</to>
  <from>Steve</from>
  <heading>Reminder</heading>
  <body>Don't forget the meeting tomorrow!</body>
</note>
```

Explanation:

XML Elements Explanation:

- `<?xml version="1.0" encoding="UTF-8"?>` → Declaration (tells version & encoding).
- `<note>` → Root element (every XML file must have **one root element**).
- `<to>`, `<from>`, `<heading>`, `<body>` → Child elements.
- Data is stored between opening `<tag>` and closing `</tag>`.

XML Rules:

- ⇒ Must have a **root element** (e.g., `<note>`).
- ⇒ Tags are **case-sensitive** (`<Name>` ≠ `<name>`).
- ⇒ Elements must be **properly closed** (`<title></title>`).
- ⇒ XML must be **well-formed** (**correct nesting**).

Example (Correct Nesting):

```
<book>
  <title>PHP Basics</title>
  <author>John Doe</author>
</book>
```

Wrong Example:

```
<book>
  <title>PHP Basics
  <author>John Doe</author>
</book>
```

XML with Attributes

⇒ Elements can have attributes (like HTML).

Example:

```
<book id="101" category="Programming">
  <title>Learning PHP</title>
  <author>Jane Smith</author>
</book>
```

- `id="101"` and `category="Programming"` are **attributes**.
- `title` and `author` are **child elements**.

Example: Library XML File

```
<library>
  <book id="1">
    <title>PHP Basics</title>
    <author>John Doe</author>
  </book>
  <book id="2">
    <title>MySQL Guide</title>
    <author>Jane Smith</author>
  </book>
</library>
```

XML vs HTML

Feature	XML	HTML
Purpose	Store/transport data	Display data (web pages)
Tags	User-defined	Predefined (<p>, <h1>)
Closing tags	Must be closed	Some can be left open
Case-sensitive	Yes	No
Used in	APIs, Config, Data sharing	Websites, UI formatting

PHP + XML (SimpleXML Read/Write)

- ⇒ A **PHP extension** that makes it **easy to read and write XML files**.
- ⇒ Treats XML elements like **objects** and **arrays**.
- ⇒ Good for **small to medium XML files**.

Reading XML with SimpleXML

- ⇒ Suppose we have a file:

library.xml	
<pre><library> <book id="1"> <title>PHP Basics</title> <author>John Doe</author> </book> <book id="2"> <title>MySQL Guide</title> <author>Jane Smith</author> </book> </library></pre>	
PHP Code to Read:	Output:
<pre><?php // Load the XML file \$xml = simplexml_load_file("library.xml") or die("Error: Cannot load file"); // Loop through each book foreach(\$xml->book as \$book) { echo "Book ID: " . \$book['id'] . "
"; echo "Title: " . \$book->title . "
"; echo "Author: " . \$book->author . "

"; } ?></pre>	Book ID: 1 Title: PHP Basics Author: John Doe Book ID: 2 Title: MySQL Guide Author: Jane Smith

Writing to XML (Create XML File)

⇒ We can also create a new XML file with PHP.

createXml.php

```
<?php
// Create XML structure
$xml = new SimpleXMLElement("<library></library>");

// Add first book
$book1 = $xml->addChild("book");
$book1->addAttribute("id", "1");
$book1->addChild("title", "PHP Basics");
$book1->addChild("author", "John Doe");

// Add second book
$book2 = $xml->addChild("book");
$book2->addAttribute("id", "2");
$book2->addChild("title", "MySQL Guide");
$book2->addChild("author", "Jane Smith");

// Save as XML file
$xml->asXML("new_library.xml");

echo "XML File created successfully!";
?>
```

Resulting new_library.xml

```
<library>
  <book id="1">
    <title>PHP Basics</title>
    <author>John Doe</author>
  </book>
  <book id="2">
    <title>MySQL Guide</title>
    <author>Jane Smith</author>
  </book>
</library>
```

Modifying Existing XML

⇒ We can edit nodes and attributes.

PHP Code to Modifying:	Output:
<pre><?php \$xml = simplexml_load_file("library.xml"); // Change the title of the first book \$xml->book[0]->title = "Advanced PHP"; // Save changes back to file \$xml->asXML("library.xml"); echo "Book updated!"; ?></pre>	Book ID: 1 Title: Advanced PHP Author: John Doe Book ID: 2 Title: MySQL Guide Author: Jane Smith

Key Functions in SimpleXML

Function	Use
<code>simplexml_load_file()</code>	Load XML file into an object
<code>simplexml_load_string()</code>	Load XML string directly
<code>\$xml->addChild()</code>	Add new child element
<code>\$xml->addAttribute()</code>	Add attribute to element
<code>\$xml->asXML()</code>	Save/export XML to file

ASYNCHRONOUS JAVASCRIPT AND XML (AJAX)

AJAX (Asynchronous JavaScript and XML)

- **AJAX = Asynchronous JavaScript and XML**
- Allows web pages to **update data in the background** without reloading the entire page.
- Uses:
 - **JavaScript** (to send requests)
 - **XMLHttpRequest** object or **Fetch API** (to communicate with server)
 - **PHP/MySQL** (server-side processing)

In short: AJAX = Send/Receive data from server without page reload.

How AJAX Works (Flow)

- ⇒ User triggers an event (e.g., button click).
- ⇒ JavaScript creates an **AJAX request**.
- ⇒ Request is sent to the **server (PHP)**.
- ⇒ PHP processes the request (maybe fetches data from DB).
- ⇒ PHP sends **response** back to JavaScript.
- ⇒ JavaScript updates the **webpage dynamically** (without refresh).

Key AJAX Functions

Term	Meaning
<code>XMLHttpRequest()</code>	Creates AJAX request
<code>xhttp.open(method, url, async)</code>	Defines request (GET/POST, target URL, async or not)
<code>xhttp.send()</code>	Sends request to server
<code>readyState</code>	Status of request (0 to 4)
<code>status</code>	HTTP status (200 = OK)

AJAX readyState Values

Value	Meaning	Description
0	UNSENT	Request is created but not yet opened. (You just made <code>new XMLHttpRequest()</code> .)
1	OPENED	<code>open()</code> method has been called. Request is ready to be sent.
2	HEADERS_RECEIVED	Request has been sent, and the server has responded with headers.
3	LOADING	The response body is being received (data is coming in).
4	DONE	The request is complete, and the response is fully available.

AJAX – GET Request

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>AJAX Example</title>
</head>
<body>
  <h2>AJAX Demo</h2>
  <button onclick="loadData()">Get Message</button>
  <p id="result"></p>

  <script>
    function loadData() {
      // Create XMLHttpRequest object
      var xhttp = new XMLHttpRequest();

      // What happens when response arrives
      xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
          document.getElementById("result").innerHTML = this.responseText;
        }
      };

      // Send request to PHP file
      xhttp.open("GET", "server.php", true);
      xhttp.send();
    }
  </script>
</body>
</html>
```

server.php

```
<?php
echo "Hello! This message is from the server using AJAX.";
?>
```

Output:

Hello! This message is from the server using AJAX.

Example with Input Data

index.html

```
<input type="text" id="name" placeholder="Enter your name">
<button onclick="sendName()">Submit</button>
<p id="greet"></p>

<script>
function sendName() {
    var name = document.getElementById("name").value;

    var xhttp = new XMLHttpRequest();
    xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
            document.getElementById("greet").innerHTML = this.responseText;
        }
    };

    xhttp.open("GET", "greet.php?name=" + name, true);
    xhttp.send();
}
</script>
```

greet.php

```
<?php
$name = $_GET['name'];
echo "Hello, " . htmlspecialchars($name) . "! Welcome to AJAX.";
?>
```

Output:

Hello, " Mohsin Welcome to AJAX

Benefits of AJAX

- Faster → No full page reload.
- Better user experience (like Gmail, Facebook chat).
- Load data **asynchronously** (in background).

AJAX – POST Request

- In AJAX, **POST** is used to **send data securely** to the server (unlike GET, data is not shown in URL).
- Useful for **forms, login, registration, saving data**.

How AJAX Works (Flow)

- ⇒ User triggers an event (e.g., button click).
- ⇒ JavaScript creates an **AJAX request**.
- ⇒ Request is sent to the **server (PHP)**.
- ⇒ PHP processes the request (maybe fetches data from DB).
- ⇒ PHP sends **response** back to JavaScript.
- ⇒ JavaScript updates the **webpage dynamically** (without refresh).

Example: Send Name using POST

index.html

```
<!DOCTYPE html>
<html>
<head>
  <title>AJAX POST Example</title>
</head>
<body>
  <h2>AJAX POST Request</h2>

  <input type="text" id="name" placeholder="Enter your name">
  <button onclick="sendData()">Submit</button>

  <p id="response"></p>

  <script>
    function sendData() {
      var name = document.getElementById("name").value;
      var xhttp = new XMLHttpRequest();

      xhttp.onreadystatechange = function() {
        if (this.readyState == 4 && this.status == 200) {
          document.getElementById("response").innerHTML = this.responseText;
        }
      };
    };
  </script>
</body>
</html>
```

```

    // Open POST request
    xhttp.open("POST", "server.php", true);

    // Send form data in POST request
    xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");
    xhttp.send("name=" + encodeURIComponent(name));
}
</script>
</body>
</html>

```

server.php

```

<?php
if (isset($_POST['name'])) {
    $name = $_POST['name'];
    echo "Hello, " . htmlspecialchars($name) . "! (via POST request)";
} else {
    echo "No name received.";
}
?>

```

Output:

Hello, Mohsin! (via POST request)

Key AJAX Functions

Step	Code
Create object	<code>var xhttp = new XMLHttpRequest();</code>
Open request	<code>xhttp.open("POST", "server.php", true);</code>
Set header	<code>xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");</code>
Send data	<code>xhttp.send("key=value");</code>
Get data in PHP	<code>\$_POST['key']</code>

XMLHttpRequest Object Methods

⇒ The `XMLHttpRequest` object in JavaScript is used to **send and receive data** from a server without reloading the page.

Commonly Used Methods:

Method	Description	Example
<code>open(method, url, async)</code>	Initializes request.	<code>xhttp.open("GET", "data.php", true);</code>
<code>send()</code>	Sends the request (for GET).	<code>xhttp.send();</code>
<code>send(string)</code>	Sends request with data (for POST).	<code>xhttp.send("name=John&age=25");</code>
<code>setRequestHeader(header, value)</code>	Adds HTTP header to request (used in POST).	<code>xhttp.setRequestHeader("Content-type", "application/x-www-form-urlencoded");</code>
<code>abort()</code>	Cancels the request.	<code>xhttp.abort();</code>
<code>getAllResponseHeaders()</code>	Returns all response headers from server.	<code>console.log(xhttp.getAllResponseHeaders());</code>
<code>getResponseHeader(headerName)</code>	Returns specific response header.	<code>console.log(xhttp.getResponseHeader("Content-Type"));</code>

AJAX Response

Property / Method	Description
<code>responseText</code>	Response as plain text (string).
<code>responseXML</code>	Response as XML document.
<code>getResponseHeader("name")</code>	Returns specific header value (e.g., "Content-Type").
<code>getAllResponseHeaders()</code>	Returns all headers as one string.

JSON

JSON (JavaScript Object Notation)

- ⇒ JSON is a **lightweight data format** used for storing and exchanging data.
- ⇒ It is easy for humans to read/write and easy for machines to parse.
- ⇒ JSON is often used in APIs and web applications.

Basic Rules of JSON

- Data is written in **key/value pairs**.
- Keys must be in **double quotes** (" ").
- Values can be: **string, number, boolean, null, array, or object**.

Example JSON

```
{
  "name": "Alice",
  "age": 25,
  "isStudent": true,
  "skills": ["HTML", "CSS", "JS"],
  "address": {
    "city": "New York",
    "zip": "1001"
  }
}
```

JSON vs JavaScript Object

JSON	JavaScript Object
Keys in double quotes	Keys can be unquoted or quoted
Data format (text only)	Actual object in memory
Can be sent over network (API)	Used only inside JavaScript code

Common JSON Methods in JavaScript

Method	Description
JSON.stringify(obj)	Converts JS object → JSON string
JSON.parse(jsonString)	Converts JSON string → JS object

JavaScript JSON – Read / Write

- ⇒ JSON = **JavaScript Object Notation**
- ⇒ Used to **store and exchange data** between client and server.
- ⇒ JSON is **text-based**, but can easily be converted to **JavaScript objects**.

Converting JS Object to JSON (Stringify)

Example JSON

```
let data = {
  name: "Alice",
  age: 25,
  skills: ["HTML", "CSS", "JS"]
};

// Convert JS object to JSON string
let jsonData = JSON.stringify(data);
console.log(jsonData);
```

Output:

```
{"name": "Alice", "age": 25, "skills": ["HTML", "CSS", "JS"]}
```

Converting JSON to JS Object (Parse)

Example JSON

```
let jsonData = '{"name": "Alice", "age": 25, "skills": ["HTML", "CSS", "JS"]}';

// Convert JSON string to JS object
let data = JSON.parse(jsonData);

console.log(data.name); // Alice
console.log(data.skills); // ["HTML", "CSS", "JS"]
```

Summary Table

Function	Use
<code>JSON.stringify(obj)</code>	Convert JS object → JSON string
<code>JSON.parse(jsonStr)</code>	Convert JSON string → JS object
<code>localStorage.setItem(key, value)</code>	Save JSON string in browser storage
<code>localStorage.getItem(key)</code>	Retrieve JSON string from storage

PHP JSON – Read / Write

⇒ PHP provides `json_encode()` and `json_decode()` functions to handle JSON.

Encoding PHP Array/Object to JSON

Example JSON

```
<?php
$data = array(
    "name" => "Alice",
    "age" => 25,
    "skills" => array("HTML", "CSS", "JS")
);

// Convert PHP array to JSON string
$jsonData = json_encode($data, JSON_PRETTY_PRINT);

echo $jsonData;
?>
```

Output:

```
{ "name": "Alice", "age": 25, "skills": [ "HTML", "CSS", "JS" ] }
```

Writing JSON to a File

Example JSON

```
<?php
file_put_contents("data.json", $jsonData);
echo "JSON file created!";
?>
```

`file_put_contents()` → saves JSON string to a file.

Reading JSON from a File

Example JSON

```
<?php
$jsonData = file_get_contents("data.json");
$data = json_decode($jsonData, true); // true → associative array

print_r($data);
?>
```


Decoding JSON

- ⇒ `json_decode($jsonString, true)` → converts JSON string → PHP **associative array**
- ⇒ `json_decode($jsonString)` → converts JSON string → PHP **object**

Key Functions Summary

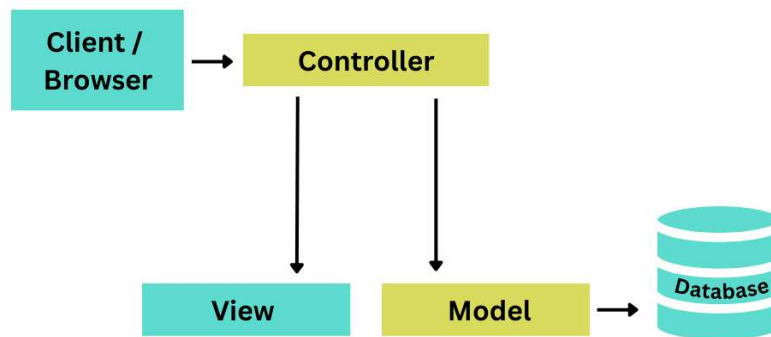
Function	Use
<code>json_encode(\$data)</code>	Convert PHP array/object → JSON string
<code>json_decode(\$json, true)</code>	Convert JSON string → PHP associative array
<code>json_decode(\$json)</code>	Convert JSON string → PHP object
<code>file_put_contents("file.json", \$data)</code>	Save JSON to file
<code>file_get_contents("file.json")</code>	Read JSON from file

MODEL-VIEW- CONTROLLER (MVC)

MVC (Model–View–Controller) in PHP

- ⇒ MVC is a **software design pattern** used to **separate application logic** into 3 parts:
- **Model** → Handles **data & business logic**.
 - **View** → Handles **UI (user interface / presentation)**.
 - **Controller** → Handles **user requests & connects Model + View**.
- ⇒ Makes applications **organized, reusable, and easier to maintain**.

MVC Structure



Components

A. Model

- Deals with **data and database interaction**.
- Contains **business logic** (rules, calculations, queries).

Example (Model.php) :

```
<?php
// Model: Store students in an array (can be DB later)
function getStudents() {
    return [
        ["id" => 1, "name" => "Mohsin", "course" => "PHP"],
        ["id" => 2, "name" => "Nihan", "course" => "MySQL"],
        ["id" => 3, "name" => "Jam", "course" => "JavaScript"]
    ];
}
?>
```

B. View

- Handles **presentation (UI/HTML)**.
- Displays data received from **Controller**.

View.php

```
<?php
// View: Display student list
function showStudents($students) {
    echo "<h2>Student List</h2>";
    echo "<table border='1' cellpadding='5'>";
    echo "<tr>
        <th>ID</th>
        <th>Name</th>
        <th>Course</th>
    </tr>";
    foreach ($students as $student) {
        echo "<tr>";
        echo "<td>" . $student['id'] . "</td>";
        echo "<td>" . $student['name'] . "</td>";
        echo "<td>" . $student['course'] . "</td>";
        echo "</tr>";
    }
    echo "</table>";
}
?>
```

C. Controller

- Acts as a **middleman** between **Model** and **View**.
- Gets user input → tells Model what to do → passes data to View.

Controller.php

```
<?php
include "model.php";
include "view.php";

// Controller: Get data from model
$students = getStudents();

// Send data to view
showStudents($students);
?>
```

→ Index (index.php)

⇒ Starting point of application.

index.php

```
<?php
// Load the controller (which will load model + view)
include "controller.php";
?>
```

output

Student List

ID	Name	Course
1	Mohsin	PHP
2	Nihan	MySQL
3	Jam	JavaScript

How MVC Works (Flow)

- **User request** (clicks button, visits URL).
- **Controller** receives request → asks Model for data.
- **Model** fetches/updates data (e.g., from database).
- **Controller** sends data to View.
- **View** displays data to user.

Benefits of MVC

- ⇒ Code is **organized & reusable**.
- ⇒ Easy to **maintain & scale** applications.
- ⇒ **Separation of concerns** (UI ≠ Logic ≠ Data).
- ⇒ Supports **teamwork** (Frontend devs handle Views, Backend devs handle Models).

PHP MYSQL DATABASE

PHP MySQL Database

- ⇒ PHP can connect to **MySQL databases** to **store, update, and retrieve data**.
- ⇒ Two common extensions:
 - **MySQLi (MySQL Improved)** – procedural or object-oriented.
 - **PDO (PHP Data Objects)** – supports multiple databases.

Connecting to MySQL Database: Using MySQLi (Procedural)

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "testdb";

$conn = mysqli_connect($servername, $username, $password, $dbname);

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully";
?>
```

Creating a Database:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "");
$sql = "CREATE DATABASE myDB";

if (mysqli_query($conn, $sql)) {
    echo "Database created successfully!";
} else {
    echo "Error: " . mysqli_error($conn);
}
mysqli_close($conn);
?>
```

Creating a Table:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "myDB");

$sql = "CREATE TABLE Users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(50),
    reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)";

if (mysqli_query($conn, $sql)) {
    echo "Table created successfully!";
} else {
    echo "Error: " . mysqli_error($conn);
}
?>
```

Insert Data:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "myDB");

$sql = "INSERT INTO Users (name, email) VALUES ('Nihna', 'Nihan@example.com')";

if (mysqli_query($conn, $sql)) {
    echo "New record inserted!";
} else {
    echo "Error: " . mysqli_error($conn);
}
?>
```


Select Data:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "myDB");

$sql = "SELECT id, name, email FROM Users";
$result = mysqli_query($conn, $sql);

if (mysqli_num_rows($result) > 0) {
    while($row = mysqli_fetch_assoc($result)) {
        echo "ID: " . $row["id"]. " - Name: " . $row["name"]. " - Email: " . $row["email"]. "<br>";
    }
} else {
    echo "0 results";
}
?>
```

mysqli_fetch_assoc():

- ⇒ It **fetches one row** from the result set **as an associative array**.
- ⇒ Keys of the array = column names in your table.
- ⇒ Each time you call it, it moves to the **next row**.
- ⇒ When no more rows are left, it returns **NULL**

Output Example:

```
ID: 1 - Name: Mohsin - Email: mohsin@example.com
ID: 2 - Name: Alif - Email: alif@example.com
ID: 3 - Name: Jam - Email: jam@example.com
```

Update Data:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "myDB");

$sql = "UPDATE Users SET email='newemail@example.com' WHERE id=1";

if (mysqli_query($conn, $sql)) {
    echo "Record updated!";
} else {
    echo "Error: " . mysqli_error($conn);
}
?>
```

Delete Data:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "myDB");

$sql = "DELETE FROM Users WHERE id=1";

if (mysqli_query($conn, $sql)) {
    echo "Record deleted!";
} else {
    echo "Error: " . mysqli_error($conn);
}
?>
```

Closing Connection:

```
<?php
mysqli_close($conn);
?>
```

It used to **close the database connection** that was opened earlier with `mysqli_connect()`.

Example of Closing Connection:

```
<?php
$conn = mysqli_connect("localhost", "root", "", "testdb");

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

echo "Connected successfully!";

// your queries go here
// ...

// close the connection
mysqli_close($conn);
?>
```

Note: If you forget to call `mysqli_close($conn)` ;, PHP will **automatically close** the connection at the end of the script execution.

PHP MySQL Database (Object-oriented)

Connecting to MySQL Database: Using MySQLi (Object-oriented)

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "testdb";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully!";
?>
```

Creating a Database:

```
<?php
$conn = new mysqli("localhost", "root", "", "");

$sql = "CREATE DATABASE myDB";
if ($conn->query($sql) === TRUE) {
    echo "Database created successfully!";
} else {
    echo "Error: " . $conn->error;
}
$conn->close();
?>
```

Creating a Table:

```
<?php
$conn = new mysqli("localhost", "root", "", "myDB");

$sql = "CREATE TABLE Users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(50),
    reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
)";

if ($conn->query($sql) === TRUE) {
    echo "Table created successfully!";
} else {
    echo "Error: " . $conn->error;
}
$conn->close();
?>
```

Insert Data:

```
<?php
$conn = new mysqli("localhost", "root", "", "myDB");

$sql = "INSERT INTO Users (name, email) VALUES ('Alif', 'alif@example.com')";

if ($conn->query($sql) === TRUE) {
    echo "New record inserted!";
} else {
    echo "Error: " . $conn->error;
}
$conn->close();
?>
```

Select Data:

```
<?php
$conn = new mysqli("localhost", "root", "", "myDB");

$sql = "SELECT id, name, email FROM Users";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    while($row = $result->fetch_assoc()) {
        echo "ID: " . $row["id"]. " - Name: " . $row["name"]. " - Email: " . $row["email"]. "<br>";
    }
} else {
    echo "0 results";
}
$conn->close();
?>
```

Delete Data:

```
<?php
$conn = new mysqli("localhost", "root", "", "myDB");

$sql = "DELETE FROM Users WHERE id=2";

if ($conn->query($sql) === TRUE) {
    echo "Record deleted!";
} else {
    echo "Error: " . $conn->error;
}
$conn->close();
?>
```

Closing the Connection:

```
<?php
$conn->close();
?>
```

PHP MySQL Database (PDO)

- **PDO (PHP Data Objects)** is a database access layer in PHP.
- Supports **many databases** (MySQL, PostgreSQL, SQLite, etc.).
- Provides **prepared statements** → safer from SQL Injection.

Connecting to MySQL Database: Using (PDO)

```
<?php
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "testdb";

try {
    $conn = new PDO("mysql:host=$servername;dbname=$dbname", $username, $password);
    // Set error mode
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
    echo "Connected successfully!";
} catch (PDOException $e) {
    echo "Connection failed: " . $e->getMessage();
}
?>
```

With Prepared Statement (Safe Code)

```
<?php
try {
    $conn = new PDO("mysql:host=localhost", "root", "");
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "CREATE DATABASE myDB_PDO";
    $conn->exec($sql);
    echo "Database created successfully!";
} catch (PDOException $e) {
    echo "Error: " . $e->getMessage();
}
$conn = null;
?>
```

Creating a Table:

```
<?php
try {
    $conn = new PDO("mysql:host=localhost;dbname=myDB_PDO", "root", "");
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "CREATE TABLE Users (
        id INT AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(50) NOT NULL,
        email VARCHAR(50),
        reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    )";

    $conn->exec($sql);
    echo "Table created successfully!";
} catch (PDOException $e) {
    echo "Error: " . $e->getMessage();
}
$conn = null;
?>
```

Insert Data:

```
<?php
try {
    $conn = new PDO("mysql:host=localhost;dbname=myDB_PDO", "root", "");
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "INSERT INTO Users (name, email) VALUES ('JAM', 'jam@example.com')";
    $conn->exec($sql);

    echo "New record inserted!";
} catch (PDOException $e) {
    echo "Error: " . $e->getMessage();
}
$conn = null;
?>
```

PHP Prepared Statements

- A **prepared statement** is a feature in PHP that allows you to **execute the same SQL statement multiple times** with different parameters.
- Provides **security against SQL Injection** because values are bound separately from the SQL query.
- Improves performance for repeated queries.

Prepared Statements using MySQLi (Object-Oriented)

Insert Example:

```
<?php
$conn = new mysqli("localhost", "root", "", "testdb");

// Prepare statement
$stmt = $conn->prepare("INSERT INTO Users (name, email) VALUES (?, ?)");

// Bind parameters (s = string, i = integer, d = double)
$stmt->bind_param("ss", $name, $email);

// Set values and execute
$name = "Mohsin";
$email = "mohsin@example.com";
$stmt->execute();

$name = "Jam";
$email = "jam@example.com";
$stmt->execute();

echo "Records inserted successfully!";
$stmt->close();
$conn->close();
?>
```


Select Example:

```
<?php
$conn = new mysqli("localhost", "root", "", "testdb");

$stmt = $conn->prepare("SELECT id, name, email FROM Users WHERE id = ?");
$stmt->bind_param("i", $id);

$id = 1;
$stmt->execute();

$result = $stmt->get_result();
while ($row = $result->fetch_assoc()) {
    echo $row["id"] . " - " . $row["name"] . " - " . $row["email"] . "<br>";
}

$stmt->close();
$conn->close();
?>
```

Prepared Statements using PDO

Insert Example:

```
<?php
try {
    $conn = new PDO("mysql:host=localhost;dbname=testdb", "root", "");
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $stmt = $conn->prepare("INSERT INTO Users (name, email) VALUES (:name, :email)");

    $stmt->bindParam(':name', $name);
    $stmt->bindParam(':email', $email);

    $name = "Nihan";
    $email = "nihan@example.com";
    $stmt->execute();

    $name = "David";
    $email = "david@example.com";
    $stmt->execute();

    echo "Records inserted successfully!";
} catch (PDOException $e) {
    echo "Error: " . $e->getMessage();
}

$conn = null;
?>
```

Output Tracing Practice

Problem1.

Write the output of the following code	Answer
<pre><?php \$value1 = 0.0; \$value2 = "0"; \$value3 = false; \$value4 = "Hello"; if (empty(\$value1) && is_string(\$value2) && ctype_alpha(\$value2)) { echo "Conditions 1 is true
"; } else { echo "Conditions 1 is false
"; } if (isset(\$value3) && empty(\$value1) && is_numeric(\$value2)) { echo "Conditions 2 is true
"; } else { echo "Conditions 2 is false
"; } if (isset(\$value4) && !empty(\$value4) && is_string(\$value3)) { echo "Conditions 3 is true
"; } else { echo "Conditions 3 is false
"; } ?></pre>	<pre>Conditions 1 is false Conditions 2 is true Conditions 3 is false</pre>

Problem2.

Write the output of the following code	Answer
<pre><?php \$var1 = 0; \$var2 = "123"; \$var3 = null; \$var4 = "World"; if (is_int(\$var1) && empty(\$var3) && is_numeric(\$var2)) { echo "Condition 1 is true
"; } else { echo "Condition 1 is false
"; } if (isset(\$var4) && ctype_alpha(\$var4) && empty(\$var4)) { echo "Condition 2 is true
"; } else { echo "Condition 2 is false
"; } if (is_bool(\$var3) (is_numeric(\$var2) && empty(\$var1))) { echo "Condition 3 is true
"; } else { echo "Condition 3 is false
"; } ?></pre>	<pre>Conditions 1 is true Conditions 2 is false Conditions 3 is true</pre>

Problem3.

Write the output of the following code	Answer
<pre><?php \$var1 = 42; \$var2 = "Hello123"; \$var3 = false; \$var4 = ""; \$var5 = "2023"; \$var6 = null; if (is_int(\$var1) && is_string(\$var2) && ctype_alpha(\$var2)) { echo "Condition 1 is true
"; } else { echo "Condition 1 is false
"; } if (isset(\$var6) && !empty(\$var5) is_bool(\$var3)) { echo "Condition 2 is true
"; } else { echo "Condition 2 is false
"; } if (is_numeric(\$var5) && is_null(\$var4) empty(\$var4)) { echo "Condition 3 is true
"; } else { echo "Condition 3 is false
"; } ?></pre>	<pre>Conditions 1 is false Conditions 2 is true Conditions 3 is true</pre>

Problem4.

Write the output of the following code	Answer
<pre><?php \$a = 0; \$b = "0"; \$c = "false"; \$d = false; \$e = null; \$f = " "; \$g = 123; if ((\$a == \$b && \$c == \$d) (\$e === null && !empty(\$f))) { echo "Condition 1 is TRUE
"; } else { echo "Condition 1 is FALSE
"; } if ((isset(\$e) xor isset(\$f)) && (\$b === \$a \$d == \$c)) { echo "Condition 2 is TRUE
"; } else { echo "Condition 2 is FALSE
"; } if ((!\$a && \$b) && (\$g % 2 == 1) && (\$c !== false)) { echo "Condition 3 is TRUE
"; } else { echo "Condition 3 is FALSE
"; } if ((empty(\$a) && !empty(\$b)) (!isset(\$z) && \$g > 100)) { echo "Condition 4 is TRUE
"; } else { echo "Condition 4 is FALSE
"; } ?></pre>	<pre>Condition 1 is TRUE Condition 2 is FALSE Condition 3 is FALSE Condition 4 is TRUE</pre>

Problem5.

Write the output of the following code	Answer
<pre><?php \$xmlData = ' <Companies> <CompanyDetails> <Name>TechVision Solutions</Name> <Founded>2010</Founded> <Headquarters>New York, USA</Headquarters> </CompanyDetails> <CompanyDetails> <Name>Innovative Systems</Name> <Founded>2005</Founded> <Headquarters>London, UK</Headquarters> </CompanyDetails> <CompanyDetails> <Name>FutureTech Dynamics</Name> <Founded>2018</Founded> <Headquarters>Berlin, Germany</Headquarters> </CompanyDetails> </Companies>'; \$xml = new SimpleXMLElement(\$xmlData); echo "Name: " . \$xml->CompanyDetails[0]->Name . "
"; echo "Founded: " . \$xml->CompanyDetails[1]->Founded . "
"; echo "Headquarters: " . \$xml->CompanyDetails[2]->Headquarters . "
"; ?></pre>	Name: TechVision Solutions Founded: 2005 Headquarters: Berlin, Germany

Problem6.

Write the output of the following code	Answer
<pre><?php \$xml = "<?xml version='1.0' encoding='UTF-8'?> <dataset> <company name='Cloudflare'> <type>public</type> <industry>IT</industry> <made_in> <city>San Francisco</city> </made_in> <dns>1.1.1.1</dns> <location country='USA'>California</location> </company> <company name='Google'> <dns server='alternate'>8.8.8</dns> </company> </dataset>"; \$obj = simplexml_load_string(\$xml); if (\$obj) { \$shortform = \$obj->company[0]; echo \$obj->company[0]['name'] . " (" . \$shortform->type . ")
"; echo \$shortform->location . ", " . \$obj->company[0]->location['country'] . "
"; echo "Made in: " . \$shortform->made_in->city . "
"; echo "DNS: " . \$obj->company[0]->dns . "
"; echo \$obj->company[1]['name'] . " DNS server: " . \$obj->company[1]->dns . "
"; } else { die("Failed to load!"); } ?></pre>	Cloudflare (public) California, USA Made in: San Francisco DNS: 1.1.1.1 Google DNS server: 8.8.8.8

Problem7.

Alice, Bob, and Charlie use the same PC for their lab performance. All of them are using the same browser to test their site <http://localhost/display.php>. Write the output of their actions based on the code given below.

Filename: display.php	Filename: Action.php
<pre><html> <head> </head> <body> <?php if (isset(\$_COOKIE['count'])) { echo "<p>You have visited " . \$_COOKIE['count'] . " times.</p>"; } else { echo "Cookie has been reset or not initialized."; } ?> <button>Refresh Page</button> </body> </html></pre>	<pre><?php if (!isset(\$_COOKIE['count'])) { setcookie('count', 1, time() + 3600, '/'); } else { setcookie('count', \$_COOKIE['count'] + 1, time() + 3*3600, '/'); } header("Location: display.php"); ?></pre>
User Actions:	Output
Alice: Opens Action.php for the first time.	You have visited 1 times.
Charlie: Opens Action.php	You have visited 2 times.
Charlie: Clicks the Refresh Page button.	You have visited 3 times.
Bob: Opens Action.php	You have visited 4 times.
Bob: Clicks the Refresh Page button.	You have visited 5 times.

Problem8.

Anik, Binu, and Chaity are testing their site at <http://localhost/cart.php> using the same browser. Write the output of their actions based on the code given below.

Filename: cart.php	Filename: add_item.php
<pre><html> <body> <?php if (isset(\$_COOKIE['cart'])) { echo "<p>Items in your cart: " . \$_COOKIE['cart'] . "</p>"; } else { echo "Your cart is empty."; } ?> <button>Add Item</button> </body> </html></pre>	<pre><?php if (!isset(\$_COOKIE['cart'])) { setcookie('cart', 1, time() + 3600, '/'); } else { setcookie('cart', \$_COOKIE['cart'] + 1, time() + 3600, '/'); } header("Location: cart.php"); ?></pre>
User Actions:	Output
Anik clicks the "Add Item" button on cart.php.	Items in your cart: 1
Binu opens add_item.php for the first time.	Items in your cart: 2
Chaity clicks the "Add Item" button twice.	Items in your cart: 4
Anik opens cart.php.	Items in your cart: 4
Binu clicks the "Add Item" button on cart.php again.	Items in your cart: 5

Problem9.

Alice, Bob, and Charlie use the same PC for their lab performance. They open a PHP page to test session tracking. The page contains a form where users can enter their name, submit it, and track visit counts. They can also reset the session. The files are as follows:

Filename: trace.php	Filename: action.php
<pre> <?php session_start(); if (!isset(\$_SESSION['count'])) { \$_SESSION['count'] = 0; } \$message = isset(\$_SESSION['message']) ? \$_SESSION['message'] : ""; unset(\$_SESSION['message']); ?> <!DOCTYPE html> <html> <head> <title>Trace Test</title> </head> <body> <h3>Visit Count: <?php echo \$_SESSION['count']; ?></h3> <form method="post" action="action.php"> Name: <input type="text" name="name">

 <button name="submit">Submit</button> <button name="reset">Reset</button> </form> <p><?php echo \$message; ?></p> </body> </html> </pre>	<pre> <?php session_start(); if (isset(\$_POST['submit'])) { \$_SESSION['count']++; \$name = \$_POST['name']; if (empty(\$name)) { \$_SESSION['message'] = "Name is required"; } else { \$_SESSION['message'] = "Hello, " . strtoupper(\$name); } } elseif (isset(\$_POST['reset'])) { \$_SESSION['count'] = 0; \$_SESSION['message'] = "Session reset"; } header("Location: trace.php"); exit(); ?> </pre>
For each event, write: Visit Count & Message displayed	Output
Alice: opens trace.php for the first time and does not click anything.	Visit Count: 0
Alice: leaves the input box empty and clicks Submit .	Visit Count: 1 Name is required
Charlie: enters "Charlie" and clicks Submit .	Visit Count: 2 Hello, Charlie
Bob: refreshes the page (no button clicked).	Visit Count: 2
Bob: clicks Reset .	Visit Count: 0 Session reset

Code Writing Practice

- ✂ **Problem1:** An HTML form is used to submit information to a server page defined in the action attribute of the form tag. The button that is to be clicked to send data to server must be 'submit' in type to perform this action. At the server side, data is received using \$_REQUEST server variable with the corresponding HTML control name as the key to that server variable.
- Create a PHP page named "registration.php", that will contain a HTML form that will have following input fields: First Name, Last Name, DOB, Gender, Phone, Email ID, Password and Confirm password.
 - First Name, Last Name, DOB, Phone and Email will be **textbox**, Gender will be **Radio Button** and Password and Confirm password will be **password in type**.
 - Make sure you have defined "showdata.php" as the value of action attribute of your HTML form.
 - Create a PHP page named "showdata.php" that receives the values submitted by your HTML form and displays the data. The data will only be shown if the user does not enter any empty data.
 - If user enters any empty data, it redirects to "registration.php".
 - Otherwise, all the data will be printed and email, password field will be stored in Session and redirect to "login.php".

registration.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Registration Form</title>
</head>
<body>
    <h2>Registration Form</h2>
    <form method="post" action="showdata.php">
        First Name: <input type="text" name="fname"><br><br>
        Last Name: <input type="text" name="lname"><br><br>
        Date of Birth: <input type="date" name="dob"><br><br>
        Gender:
        <input type="radio" name="gender" value="Male"> Male
        <input type="radio" name="gender" value="Female"> Female <br><br>
        Phone: <input type="text" name="phone"><br><br>
        Email ID: <input type="text" name="email"><br><br>
        Password: <input type="password" name="password"><br><br>
        Confirm Password: <input type="password" name="cpassword"><br><br>

        <input type="submit" value="Submit">
    </form>
</body>
</html>
```

showdata.php

```
<?php
session_start();

$fname      = $_REQUEST['fname'];
$lname      = $_REQUEST['lname'];
$dob        = $_REQUEST['dob'];
$gender     = $_REQUEST['gender'] ?? "";
$phone      = $_REQUEST['phone'];
$email      = $_REQUEST['email'];
$password   = $_REQUEST['password'];
$cpassword  = $_REQUEST['cpassword'];

if (empty($fname) || empty($lname) || empty($dob) || empty($gender) ||
    empty($phone) || empty($email) || empty($password) || empty($cpassword)) {
    header("Location: registration.php");
    exit;
}

if ($password !== $cpassword) {
    echo "Passwords do not match. <a href='registration.php'>Try Again</a>";
    exit;
}

$_SESSION['email'] = $email;
$_SESSION['password'] = $password;

echo "<h2>Registration Details</h2>";
echo "First Name: $fname <br>";
echo "Last Name: $lname <br>";
echo "Date of Birth: $dob <br>";
echo "Gender: $gender <br>";
echo "Phone: $phone <br>";
echo "Email: $email <br>";

header("Location: login.php");

?>
```


✂ **Problem2:** An HTML form is used to collect user details and submit them to a server page specified in the action attribute of the `<form>` tag. The button used for submission must be of type submit. On the server side, data is retrieved using the `$_POST` server variable with the corresponding HTML control names as keys.

- **Create a PHP page named `signup.php`** and it should contain an HTML form with the following input fields: **Full Name:** Textbox, **Username:** Textbox, **Password:** Password fields, **Gender:** Radio buttons (Male, Female, Other), **Country:** Dropdown with at least 5 options, **Profile Picture:** File input for image upload. The action attribute of the form must be set to "validate.php".
- **Create a PHP page named `validate.php`** to Validate the data received. This page will check if none of the given output is not NULL. If any one of input is null, then it will show a message "Enter Valid data". If user gives valid data, then display the user's given data and Save the Full Name, Username, and uploaded image path in `$_SESSION` and Redirect the user to `dashboard.php`.
- **Create a PHP page named `dashboard.php`** and display all the user information.

signup.php

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<head>
    <title>Signup</title>
</head>
<body>
    <h2>Signup Form</h2>
    <form action="validate.php" method="post" enctype="multipart/form-data">
        Full Name: <input type="text" name="fullname"><br><br>
        Username: <input type="text" name="username"><br><br>
        Password: <input type="password" name="password"><br><br>

        Gender:
        <input type="radio" name="gender" value="Male"> Male
        <input type="radio" name="gender" value="Female"> Female
        <input type="radio" name="gender" value="Other"> Other <br><br>

        Country:
        <select name="country">
            <option value="">--Select--</option>
            <option value="Bangladesh">Bangladesh</option>
            <option value="India">India</option>
            <option value="USA">USA</option>
            <option value="UK">UK</option>
            <option value="Canada">Canada</option>
        </select><br><br>

        Profile Picture: <input type="file" name="profile"><br><br>

        <input type="submit" value="Signup">
    </form>
</body>
</html>
```

validate.php

```
<?php
session_start();

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $fullname = $_POST['fullname'];
    $username = $_POST['username'];
    $password = $_POST['password'];
    $gender = $_POST['gender'];
    $country = $_POST['country'];
    $profilePic = $_FILES['profile']['name'];

    if (empty($fullname) || empty($username) || empty($password) ||
        empty($gender) || empty($country) || empty($profilePic)) {
        echo "Enter Valid data";
        exit;
    }

    $_SESSION['fullname'] = $fullname;
    $_SESSION['username'] = $username;
    $_SESSION['profile'] = $profilePic;
    $_SESSION['gender'] = $gender;
    $_SESSION['country'] = $country;

    header("Location: dashboard.php");
    exit;
} else {
    echo "Invalid Request";
}
?>
```

dashboard.php

```
<?php
session_start();
?>
<!DOCTYPE html>
<html>
<head>
    <title>Dashboard</title>
</head>
<body>
    <h2>User Dashboard</h2>
    <p>Full Name:<?php echo $_SESSION['fullname']; ?></p>
    <p>Username:<?php echo $_SESSION['username']; ?></p>
    <p>Gender:<?php echo $_SESSION['gender']; ?></p>
    <p>Country:<?php echo $_SESSION['country']; ?></p>
    <p>Profile Picture:</p>
    
</body>
</html>
```

✂ **Problem3** A web application requires a **login system** where user credentials are verified using a **MySQL database**. The login form submits user data to a server-side PHP script using the **POST** method. User authentication must be handled using PHP sessions.

- Create a PHP page named **login.php** that contains an HTML form with the following input fields: **Username**: Text box; **Password**: Password field; **Submit button** (type must be submit). The form must use the **POST** method and the action attribute must be set to **check_login.php**.
- Create a PHP page named **check_login.php** that performs the following tasks:
 1. Establish a database connection using **mysqli**
 2. Retrieve form data using the **\$_POST** variable
 3. Validate that the username and password fields are not empty
 4. Verify the user credentials from the **users** table of the **student_db** database having the columns **id**, **username**, and **password**

If login is successful, store the username in **\$_SESSION** and redirect the user to **home.php**. Otherwise, **display an appropriate error message**.

- Create a PHP page named **home.php** that retrieves data from **\$_SESSION**, displays a welcome message with the username, and provides a logout option. Use **session_start()** where necessary.

login.php

```
<!DOCTYPE html>
<html>
<head>
    <title>Login</title>
</head>
<body>
    <form method="POST" action="check_login.php">
        <label>Username:</label>
        <input type="text" name="username"><br><br>

        <label>Password:</label>
        <input type="password" name="password"><br><br>

        <input type="submit" value="Login">
    </form>
</body>
</html>
```

check_login.php

```
<?php
session_start();

$conn = mysqli_connect("localhost", "root", "", "student_db");

if (!$conn) {
    die("Database connection failed");
}

$username = $_POST['username'];
$password = $_POST['password'];

if (empty($username) || empty($password)) {
    echo "Enter valid login information";
    exit();
}

$sql = "SELECT *
        FROM users
        WHERE username='$username' AND password='$password'";
$result = mysqli_query($conn, $sql);

if (mysqli_num_rows($result) == 1) {
    $_SESSION['username'] = $username;
    header("Location: home.php");
} else {
    echo "Invalid username or password";
}
?>
```

home.php

```
<?php
session_start();
?>

<!DOCTYPE html>
<html>
<head>
    <title>Home</title>
</head>
<body>
    <h2>Welcome, <?php echo $_SESSION['username']; ?></h2>

    <a href="logout.php">Logout</a>
</body>
</html>
```

✂ **Problem5:** A web application of **AIUB** requires displaying **course information dynamically** without reloading the web page. The data must be requested from the server using **AJAX** and processed using **PHP**. The server will return the information in **JSON format**, which will be displayed on the web page.

- Create an **HTML page named** `course.html` that contains:
 - A button to load course information
 - A `<div>` element to display the course details
- Create a **JavaScript file named** `course_script.js` that contains:
 - JavaScript code using `XMLHttpRequest` to send an AJAX request to a PHP file when the button is clicked
 - The AJAX request must **not reload the page**.
- Create a **PHP file named** `get_course.php` that performs the following tasks:
 - Prepare course information for AIUB (Course Code, Course Title, Credits)
 - Store the data in an associative array
 - Encode the data into **JSON format** using `json_encode()`
 - Send the JSON response to the client
- Using JavaScript, retrieve the JSON response and dynamically display the following information inside the `<div>`:
 - Course Code: CSC 3215
 - Course Title: Web Technologies
 - Credits: 3

course.html

```
<!DOCTYPE html>
<html>
<head>
  <title>AIUB Course Information</title>
</head>
<body>

  <h3>AIUB Course Information</h3>

  <button onclick="loadCourse()">Load Course</button>

  <div id="courseInfo"></div>

  <script src="course_script.js"></script>

</body>
</html>
```

course_script.js

```
function loadCourse() {  
    var xhr = new XMLHttpRequest();  
    xhr.open("GET", "get_course.php", true);  
  
    xhr.onreadystatechange = function () {  
        if (xhr.readyState === 4 && xhr.status === 200) {  
            var data = JSON.parse(xhr.responseText);  
  
            document.getElementById("courseInfo").innerHTML =  
                "Course Code: " + data.code + "<br>" +  
                "Course Title: " + data.title + "<br>" +  
                "Credits: " + data.credits;  
        }  
    };  
  
    xhr.send();  
}
```

get_course.php

```
<?php  
$course = array(  
    "code" => "CSC 3215",  
    "title" => "Web Technologies",  
    "credits" => 3  
);  
  
echo json_encode($course);  
?>
```

Analytical Questions Practice

Problem1.

```
<!DOCTYPE html>
<html>
<body>
  <form action="process.php" method="post">
    User Name
    <input type="text" id="username" name="uname">
    Password
    <input type="text" id="password" name="pass">
    <br>
    <input type="submit" value="Submit">
  </form>
</body>
</html>
```

Write a PHP code to validate the HTML form based in Process.php page on following criteria;

- Username must not contain numbers
- Password must be more than 6 characters
- Both fields are required

If the above criteria does not meet then a message will appear **"Please fill all the fields"**.

process.php

```
<?php
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    $username = $_POST['uname'];
    $password = $_POST['pass'];

    if (empty($username) || empty($password)) {
        echo "Please fill all the fields";
    }

    elseif (preg_match('/[0-9]/', $username)) {
        echo "Username must not contain numbers";
    }

    elseif (strlen($password) <= 6) {
        echo "Password must be more than 6 characters";
    }
    else {
        echo "Form submitted successfully!";
    }
}
?>
```

Problem2.

```
<department>
  <name>Computer Science</name>
  <program>
    <name>BSc Computer Science</name>
    <course>
      <code>CSC 3215</code>
      <title>Web Technologies</title>
      <credits>3</credits>
    </course>
  </program>
</department>
```

Write a PHP code to generate XML data like given on right. After generating XML, the data will be written on “myxml.xml” file.

writexml.php

```
<?php
$xml = new DOMDocument("1.0", "UTF-8");
$xml->formatOutput = true;

$department = $xml->createElement("department");
$xml->appendChild($department);

$name = $xml->createElement("name", "Computer Science");
$department->appendChild($name);

$program = $xml->createElement("program");
$department->appendChild($program);

$programName = $xml->createElement("name", "BSc Computer Science");
$program->appendChild($programName);

$course = $xml->createElement("course");
$program->appendChild($course);

$code = $xml->createElement("code", "CSC 3215");
$title = $xml->createElement("title", "Web Technologies");
$credits = $xml->createElement("credits", "3");

$course->appendChild($code);
$course->appendChild($title);
$course->appendChild($credits);

$xml->save("myxml.xml");

echo "XML file generated successfully!";
?>
```


MCO Practice

1. Which HTTP status code indicates a successful request?
a) 100
b) 200
c) 300
d) 400
2. How to collect data using PHP super global when the form is submitted using the "GET" method?
a) \$_GET[]
b) \$_POST[]
c) \$_REQUEST[]
d) Both a & c
3. Find the correct answer to invoke a method in PHP?
a) \$object->methodName();
b) object->methodName();
c) object::methodName();
d) \$object::methodName();
4. Which of the following defines an associative array in PHP?
a) \$array = myarray("name" => "John", "age" => 25);
b) \$array = array("name" => "John", "age" => 25);
c) \$array = ["name" = "John", "age" = 25];
d) \$array = array("John", "25");
5. What is the default port for HTTP traffic?
a) 81
b) 80
c) 443
d) 8080
6. Which operator is used for concatenating strings in PHP?
a) + b) & **c) .** d) #
7. What is the main difference between HTML and XHTML?
a) HTML is stricter than XHTML
b) XHTML is an extension of JavaScript
c) XHTML is a stricter version of HTML
d) HTML is not supported by web browsers
8. Which function is used to check if a string matches a specified pattern in PHP?
a) pattern_match()
b) str_match()
c) preg_match()
d) match_string()

9. Which HTTP request method is used to retrieve data from a server?
- a) GET
 - b) POST
 - c) PUT
 - d) DELETE
10. The <head> section of an HTML document is used to
- a) Display content
 - b) Define metadata
 - c) Structure page layout
 - d) Define DOM
11. What does DHTML stand for?
- a) Dynamic Hyper Text Markup Language
 - b) Direct Hyper Text Markup Language
 - c) Dynamic High Text Markup Language
 - d) Document Hyper Text Markup Language
12. Which of the following is an XML naming rule?
- a) Names can start with a number
 - b) Names cannot contain spaces
 - c) Names can start with a punctuation character
 - d) Names cannot use any alphabetic character
13. What does the GET HTTP request method do?
- a) Deletes data from the server
 - b) Sends data to be processed to a server
 - c) Retrieves data from the server
 - d) Updates existing server data
14. Which organization is responsible for establishing web standards?
- a) IETF
 - b) W3C
 - c) HTML5 Consortium
 - d) ISO
15. In XHTML, all elements must be
- a) uppercase
 - b) lowercase
 - c) bold
 - d) italic
16. In a URL, what does the 'path' specify?
- a) The domain name
 - b) The protocol
 - c) The file location on the server
 - d) The fragment identifier

17. **What feature makes DHTML different from HTML?**
- a) Static content
 - b) Dynamic interactivity**
 - c) Hypertext links
 - d) Structured formatting
18. **In a client-server model, who initiates the request?**
- a) Server
 - b) Client**
 - c) Peer
 - d) Router
19. **URI stands for**
- a) Universal Resource Identify
 - b) Uniform Resource Identifier**
 - c) Unified Resource Identify
 - d) Universe Resource Identifier
20. **Which function checks the data type and value in PHP?**
- a) print_r()
 - b) var_dump()**
 - c) gettype()
 - d) checkType()
21. **What data types does PHP support for decimal values?**
- a) Integer
 - b) Double
 - c) Float**
 - d) Boolean
22. **What is the purpose of isset() in PHP?**
- a) To declare variables
 - b) To check if a variable is defined**
 - c) To compare two values
 - d) To print output
23. **What keyword is used to create a function in PHP?**
- a) variable
 - b) function**
 - c) new
 - d) declare
24. **How are associative arrays defined in PHP?**
- a) With indexes starting at 0
 - b) With key-value pairs**
 - c) With numeric keys
 - d) With string keys only

25. Which superglobal in PHP can access global variables from anywhere?
- a) \$_GET
 - b) \$_SESSION
 - c) \$GLOBALS
 - d) \$_ENV
26. What does the is_numeric() function check in PHP?
- a) If a variable is numeric or a numeric string
 - b) If a variable is a valid string
 - c) If a variable is an array
 - d) If a variable is NULL
27. What is the difference between GET and POST in PHP?
- a) GET displays data in the URL, POST does not
 - b) POST is faster than GET
 - c) GET is more secure than POST
 - d) POST is limited to 100 characters
28. What is the purpose of the \$_REQUEST superglobal in PHP?
- a) Collects data from POST, GET, and COOKIE
 - b) Handles session variables
 - c) Sets server configuration
 - d) Collects environment variables
29. To check if a variable is an integer in PHP, you can use
- a) is_integer()
 - b) is_string()
 - c) isset()
 - d) is_numeric()
30. What attribute is necessary for file upload in HTML forms?
- a) method="get"
 - b) enctype="multipart/form-data"
 - c) action="file_upload"
 - d) method="upload"
31. Which one of the following HTTP methods should be used while sending passwords or other sensitive information?
- a) GET
 - b) POST
 - c) PUT
 - d) NEXT
32. SESSION & COOKIE both are PHP super global variables used to store data in client machine.
- a) True
 - b) False

33. **PHP stands for:**
- a) PHP: Hypertext Processor
 - b) Personal Home Page
 - c) Both i & ii
 - d) PHP: Hypertext Preprocessor
34. **Which one of the following is the right way of defining a function in PHP?**
- a) `function functionName(parameters) { function body }`
 - b) `functionName(parameters) { function body }`
 - c) `function { functionName(parameters) }`
 - d) `function { function body }`
35. **What does the trim() function in PHP do?**
- a) Removes white spaces from both sides of a string
 - b) Validates strings
 - c) Removes characters from one string
 - d) None of the above
36. **Which function is used to include the contents of one PHP file into another?**
- a) `include()`
 - b) `header()`
 - c) `import()`
 - d) `append()`
37. **In the URL "<https://www.example.com/page.html?id=123>", what does "id=123" represent?**
- a) URL encoding
 - b) Authority
 - c) Query Parameter
 - d) Domain Name
38. **Which one of the following statements is false about PHP form handling?**
- a) The amount of data that can be submitted from a form using GET method is limited
 - b) The HTTP GET method should never be used for sending passwords or other sensitive information
 - c) The PHP super globals `$_GET` and `$_POST` is used to collect form data.
 - d) The HTTP POST requests are never cached, and can never bookmark them.
39. **Which one of the following functions is used to return the length of an array?**
- a) `count()`
 - b) `length()`
 - c) `len()`
 - d) `size()`
40. **Which one of the following statements is false about PHP sessions?**
- a) Session data in PHP is stored on the client side by default.
 - b) By default, session variables last until the user closes the browser.
 - c) Session variables hold information about one single user and are available to all pages in one application.
 - d) None of the above.

41. Which one of the following is true about the id attribute in HTML?
- a) An element can have multiple id values
 - b) The id attribute must be unique within a single HTML document.**
 - c) The id attribute is used only with form elements.
 - d) The id attribute is optional and cannot be used for styling.
42. In PHP, the is_bool() function checks if a variable is:
- a) 1 or 0
 - b) true or false**
 - c) NULL
 - d) empty
43. What type of data does \$_FILES in PHP handle?
- a) String data
 - b) Form data
 - c) Uploaded file data**
 - d) Numeric values
44. PHP's strlen() function is used to:
- a) Find the length of an integer
 - b) Find the length of a string**
 - c) Convert a string to uppercase
 - d) Validate numbers
45. In PHP, how do you check the file size of an uploaded file?
- a) \$_FILES['file']['size']**
 - b) \$_FILES['file']['length']
 - c) file_size(\$_FILES['file'])
 - d) size(\$_FILES['file'])
46. How do you retrieve a session variable in PHP?
- a) \$_COOKIE['variable_name']
 - b) \$_SESSION['variable_name']**
 - c) \$_SERVER['variable_name']
 - d) \$_FILES['variable_name']
47. In PHP, what function must appear before any HTML tags when starting a session?
- a) setcookie()
 - b) session_start()**
 - c) session_init()
 - d) session_set()
48. Which of the following PHP statement/statements will store 111 in variable num?
- a) int \$num = 111; b) 111 = \$num; c) int mum = 111; **d) \$num = 111;**

49. What is the main purpose of a cookie in web applications?

- a) to store temporary data in the server
- b) to track user sessions across the multiple pages
- c) to identify a user by storing small files on the client computer
- d) to manage server configurations

50. How do you create a session variable in PHP?

- a) `$_COOKIE['variable_name']`
- b) `$_SESSION['variable_name']`
- c) `$_SERVER['variable_name']`
- d) `$_FILES['variable_name']`

51. Which one of the following statements is false about PHP sessions?

- a) Session data in PHP is stored on the client side
- b) By default, session variables last until the user closes the browser.
- c) Session variables hold information about one single user and are available to all pages in one application.
- d) None of the above.

52. Which one of the following statements is false about PHP form handling?

- a) The amount of data that can be supplied from a form using the POST method is limited
- b) The HTTP GET method should never be used for sending passwords or other sensitive information.
- c) The PHP super global `$_GET` and `$_POST` is used to collect form data.
- d) The HTTP POST requests are never cached, and we cannot bookmark them.

53. Which one of the following should not be used while sending passwords or other sensitive information?

- a) GET
- b) POST
- c) PUT
- d) NEXT

54. What attribute is necessary for file upload in HTML forms?

- a) `method="get"`
- b) `method="upload"`
- c) `enctype="multipart/form-data"`
- d) `action="file_upload"`

55. What does PHP stand for?

- a) Personal Home Page
- b) PHP: Hypertext Preprocessor
- c) Preprocessed Hypertext Page
- d) Private Hypertext Protocol

56. Where are PHP scripts executed?

- a) Client-side (browser)
- b) Server-side**
- c) Both client and server
- d) Mobile OS

57. Which PHP superglobal variable holds information about headers, paths, and script locations?

- a) \$_GET
- b) \$_POST
- c) \$_SERVER**
- d) \$_REQUEST

58. How do you access the "name" field submitted via a form using the POST method?

- a) \$_POST["name"]**
- b) \$_GET["name"]
- c) \$_REQUEST["name"]
- d) \$form["name"]

59. Which of the following is an example of an associative array?

- a) \$cars = ["Volvo", "BMW", "Toyota"];
- b) \$car = ["brand" => "Ford", "model" => "AMG"];**
- c) \$cars = array("Volvo", "BMW", "Toyota");
- d) \$car = array("Ford", "Mustang");

60. What does the \$_REQUEST superglobal contain?

- a) Only GET data
- b) Only POST data
- c) Data from GET, POST**
- d) Only session data

61. What is the output of following PHP code?

```
$x = "10";  
$y = 20;  
echo $x + $y;
```

- a) 1020
- b) 30**
- c) "10" + 20
- d) Undefined

62. If \$var variable is not declared then isset(\$var) will return?

- a) true
- b) false**

63. `echo strlen("\n WebTechIsFun Learning;")` will return?
- a) 24
 - b) 23
 - c) 22
 - d) 21
64. To check if a user has submitted a value in an input form field use the ____ PHP function.
- a) `empty()`
 - b) `isEmpty()`
 - c) `set()`
 - d) `isset()`
65. To check if a variable is an integer in PHP, you can use ____ .
- a) `is_integer()`
 - b) `is_string()`
 - c) `isset()`
 - d) `is_numeric()`
66. To check the length of an array in PHP use ____ function.
- a) `count()`
 - b) `length()`
 - c) `len()`
 - d) `size()`
67. Which attribute is used to specify the HTTP method used to submit the form data in PHP?
- a) `id`
 - b) `method`
 - c) `name`
 - d) `action`
68. `echo (is_bool(true) == is_bool(false));` in PHP will return ____ .
- a) true
 - b) 1
 - c) empty
 - d) null
69. A developer needs to store a user's language preference persistently for 30 days, even if they close the browser. Which is the best approach?
- a) Store it in `$_SESSION`.
 - b) Use browser `localStorage`.
 - c) Set a cookie with expires set to 30 days.
 - d) Save it in a database.
70. A user clicks "Logout," but their preferences cookie remains stored in the browser. How can the developer ensure the cookie is deleted upon logout?
- a) Use `unset($_COOKIE['preferences'])`
 - b) Use `setcookie('preferences', '', time()-3600)`
 - c) Clear the browser cache.
 - d) Set the cookie path to `/logout`.

71. A user logs into a website, and their login status is stored in a PHP session. However, after navigating to another page, the session data is lost. What could be the most likely reason for this issue?
- a) The browser does not support cookies.
 - b) The server's disk space is full.
 - c) The `session_start()` function was not called at the beginning of each PHP script.
 - d) The session data was manually deleted using `session_destroy()`.
72. A session stores a shopping cart (`$_SESSION['cart']`). A cookie remembers the user's city (`user_city=NYC`). If the user clears cookies but not session data, what happens?
- a) The cart remains; city is lost.
 - b) The cart is lost; city remains.
 - c) Both are lost.
 - d) Neither is affected.
73. Logout script does two things:
- Step 1: Ends the session with `session_destroy()`
- Step 2: Deletes a cookie (`setcookie('remember_me','', time()-3600)`).
- What happens if only Step 1 is executed?
- a) The user stays logged in.
 - b) The cookie keeps the session alive.
 - c) The session is destroyed, but the cookie remains unused.
 - d) The browser deletes both automatically.
74. Why is MVC better than messy PHP code?
- a) Mixes HTML, PHP, and SQL together
 - b) Puts all logic in the View
 - c) Keeps code organized and easier to manage
 - d) Doesn't need databases
75. A PHP script receives this URL: `http://localhost/welcome.php?key=xyz`. Which of the following syntax can be used to retrieve the query value?
- a) `$_GET['key']`
 - b) `$_REQUEST['key']`
 - c) Both a & b
 - d) `$_POST['key']`
76. A user attempts to upload a profile picture through a PHP form, but the server receives only file details like size and type while the actual image data is missing. Which is the most likely required limit in the form to ensure the file content is transmitted?
- a) Increase the `max_execution_time`
 - b) Increase the PHP memory limit in `php.ini`
 - c) Add `enctype="multipart/form-data"`
 - d) Add `method="get"`

77. What is the difference between the `move_uploaded_file()` function and a regular `copy()` function when handling uploaded files in PHP?
- a) `move_uploaded_file()` requires the file to be uploaded via HTTP POST, ensuring security
 - b) `copy()` automatically verifies the MIME type
 - c) Both functions are identical in their functionality
 - d) None of the above
78. You are maintaining a large PHP application that loads a critical configuration file before executing the main script. You noticed that if the configuration file is missing or its path is incorrect, the script should never proceed because it depends on constants defined in that file. Which of the following choices best explains why you would choose `require` over `include` in this scenario?
- a) `require` throws a fatal error and halts script execution if the file is missing, ensuring the rest of the script cannot run.
 - b) `require` loads the file only once, whereas `include` may load it multiple times.
 - c) `require` automatically caches the included file for better performance compared to `include`.
 - d) `require` allows relative paths, whereas `include` requires absolute paths.
79. Which PHP function should be used to ensure that file includes use relative paths, whereas includes using absolute paths are automatically resolved from the server root?
- a) `__FILE__`
 - b) `__CLASS__`
 - c) `__NAMESPACE__`
 - d) `__DIR__`
- 80.
- a) `__DIR__`
 - b) `__NAMESPACE__`
 - c) `__FILE__`
 - d) `__CLASS__`
81. In PHP, a function sometimes returns a string, other times an array, or false on failure. Which statement best explains why this is valid?
- a) PHP functions must always return the same type
 - b) Strict functions require mixed return type
 - c) PHP functions can return any type unless a strict return type is specified
 - d) PHP functions require mixed return type declaration

82. **PHP functions can only return scalar values by default.**
a) True
b) False
83. **A PHP script must access form data sent via POST and the current script's filename without using global or predefined variables. Which superglobals provide this information?**
a) `$_GET` and `$_ENV`
b) `$_POST` and `$_SERVER`
c) `$_POST` and `$_ENV`
d) `$_GET` and `$_SERVER`
84. **Which PHP function returns only the values of an associative array while discarding its keys?**
a) `array_keys()`
b) `array_values()`
c) `array_column()`
d) `array_flip()`
85. **You want to check if a form field exists and whether it contains a non-empty value. Which combination of PHP functions is most appropriate?**
a) `isset()` to check existence, `empty()` to check for non-empty value
b) `empty()` to check existence, `isset()` to check non-empty value
c) Only `isset()` is sufficient
d) Only `empty()` is sufficient
86. **You need to store a user's login state: one method should persist across browser restarts, and one should last only until the browser is closed. Which statement correctly distinguishes between cookies and sessions?**
a) Cookies store data on the client and can persist; sessions store data on the server and expire on browser close.
b) Sessions store data on the client and can persist; cookies store data on the server and expire on browser close.
c) Cookies are stored in memory, while sessions are stored in files permanently.
d) Both behave the same.
87. **Which approach best prevents SQL injection while allowing PHP scripts to fetch user data from a MySQL database?**
a) Escaping input before including in SQL queries.
b) Concatenating user input directly into SQL queries.
c) Using `htmlspecialchars()` before SQL statements.
d) Using parameterized queries with prepared statements.

88. Which port number does MySQL use by default for client-server connections?
- a) 1521
 - b) 1433
 - c) 3306
 - d) 8080
89. Which PHP function is used to count the number of elements in an array?
- a) `count()`
 - b) `sizeof()`
 - c) `array_length()`
 - d) `json_encode()`
90. Which of the following MySQL procedural style functions returns the number of affected rows after an INSERT or UPDATE?
- a) `mysqli_num_rows()`
 - b) `mysqli_query_rows()`
 - c) `mysqli_affected_rows()`
 - d) `mysqli_fetch_rows()`
91. `file_uploads` in `PHP.ini` must be set to true for file uploads. Which directive must also be large enough to allow file uploads?
- a) `max_execution_time`
 - b) `memory_limit`
 - c) `upload_max_filesize`
 - d) `post_max_size`
92. In PHP, if a cookie is set without specifying an expiration time, what will happen?
- a) It will be stored permanently until manually deleted.
 - b) It will expire immediately.
 - c) It will last only for the browser session.
 - d) It will default to a 24-hour expiration.
93. Which statement correctly explains the difference between `session_unset()` and `session_destroy()` in PHP?
- a) `session_unset()` removes all session variables; `session_destroy()` deletes the session cookie and ends the session completely.
 - b) Both functions do the same thing.
 - c) `session_destroy()` removes all variables, `session_unset()` deletes the session file.
 - d) `session_unset()` deletes session variables, `session_destroy()` removes only one.

94. Which PHP function creates an associative array by using keys from one array and values from another?

- a) `array_map()`
- b) `array_merge()`
- c) `array_combine()`
- d) `array_values()`

95. What is the output of the following code?

```
$arr = ["a"=>100, "b"=>200];  
echo json_encode($arr);
```

- a) `[100,200]`
- b) `{"a":100,"b":200}`
- c) `[ "a"=>100, "b"=>200]`
- d) `100,200`

96. Which index in `$_FILES["doc"]` gives the error code of uploaded file.

- a) `$_FILES["doc"]["size"]`
- b) `$_FILES["doc"]["type"]`
- c) `$_FILES["doc"]["name"]`
- d) `$_FILES["doc"]["error"]`

Lecture Video Link

1. Web Tech Final Term (Part 1): <https://youtu.be/k6xy893fs0I?si=Gsm5k1eU2fOBbBAD>
2. Web Tech Final Term (Part 2): https://youtu.be/dlPOsS5TG_Q?si=O1AFqBdy1xEVIHKQ

Project Link

1. Project Link: <https://github.com/MIHMahmudEli/StudyHub.git>

Web Side Link

1. Project Web Side Link: <https://studyhubb.great-site.net>